In [174]:
```
from IPython.display import display, HTML

display(HTML("""
<style>
/* JupyterLab: center notebook and leave margins */
.jp-NotebookPanel-notebook {
  width: 85% !important;
  margin-left: auto !important;
  margin-right: auto !important;
  max-width: 1100px !important;       /* optional cap */
}

/* Make output area match nicely */
.jp-OutputArea {
  max-width: 100% !important;
}
</style>
"""))
# make high-resolution of images.
%config InlineBackend.figure_format = 'svg'
print("setup jnk layout and figure format.")
```

```
setup jnk layout and figure format.
```

Table of Contents

- 0. Preparing Your Environment
  - 0.1 Install Anaconda and setup environment
  - 0.2 Checking your device
- 1. Explore LLMs via ollama
  - 1.1 Install ollama and download LLMs
  - 1.2 Get response for a given prompt
  - 1.3 Get response probability from LLMs
- 2. Basics for Python and spaCy
  - 2.1 Unicode for text data
  - 2.2 Python basics: regular expression
  - 2.3 (Old way) Tokenization tool: spaCy
- 3. Datasets Exploration
  - 3.1 Heap's Law
  - 3.2 Explore wikitext-2 dataset
  - 3.3 Explore wikitext-103 dataset
  - 3.4 BookCorpus datasets
- 4. LLMs tokenization (Modern Way)
  - 4.1 Subword tokenization: BPE
  - 4.2 Subword tokenization: WordPiece
  - 4.3 Subword tokenization: Unigram
- 5. Minimum Edit Distance
  - 5.1 Algorithm analysis
  - 5.2 Algorithm implementation
- 6. Submit Your Work
- 7. Useful Commands and References

0. Preparing Your Environment

0.1 Install Anaconda and setup environment

> If you are using macOS or Linux, you are ready to go. ⚠️ **If you are using Windows, I strongly recommend installing Ubuntu 22.04 via WSL (https://ubuntu.com/desktop/wsl). In the rest of the instructions, I assume you are using Ubuntu/Linux or macOS.**

- **Step 1: Install Anaconda.** Download and install Anaconda from: https://www.anaconda.com/download (https://www.anaconda.com/download).

- **Step 2: Setup your git.** Make sure you have git (https://git-scm.com/install/) and ssh in your system (macOS usually already has both git and ssh). After the above step, you are ready to download our course github project. In your Ubuntu/Linux system, install git and sssh via

  ```
  sudo apt update
  ```

  ```
  sudo apt install -y git openssh-client
  ```

  Then, generate an SSH key (ed25519)

  ```
  ssh-keygen -t ed25519 -C "your_email@example.com"
  ```

  Press Enter for the default location, and optionally set a passphrase. This creates:
  - ~/.ssh/id_ed25519 (private key)
  - ~/.ssh/id_ed25519.pub (public key)

  Then, you can start ssh-agent and add the key:

  ```
  eval "$(ssh-agent -s)"
  ```

  ```
  ssh-add ~/.ssh/id_ed25519
  ```

  Add the public key to GitHub, that is, copy the generated key:

  ```
  cat ~/.ssh/id_ed25519.pub
  ```

  It will be something like, `ssh-ed25519 XXXXXXX...XXXXXXX your-email@xxx.com` . **Copy this line**, and then in GitHub: **Settings** → **SSH and GPG keys** → **New SSH key** → **paste.** To test the SSH connection, you can use `ssh -T git@github.com` , if it says something like "Hi USERNAME! …", you're good. After these steps, you are ready to clone our course github via

  ```
  git clone git@github.com:baojian/llm-26.git
  ```

  After the above steps, you are ready to create our course env.

- **Step 3: Create and activate the course environment.** Make sure you have installed the conda environment.
  - **Option 1:** If you have GPU in your machine (Linux + NVIDIA GPU), please create your env via

    ```
    cd llm-26/lecture-01-tokenization/ # make sure your are in our course folder.
    ```

    ```
    conda env create -f environment-gpu.yml
    ```

    Activate it with:

    ```
    conda activate llm-26-gpu
    ```

    Note: environment-gpu.yml uses `pytorch-cuda=12.1` . If your GPU driver/CUDA setup does not support CUDA 12.1, change this version accordingly (e.g., 11.8).
  - **Option 2:** If you use macOS / Windows (WSL) / Linux CPU-only, please create your env via

```
conda env create -f environment.yml
```

Activate it with:

```
conda activate llm-26-cpu
```

Note, if you got errors when installing `en_core_web_sm` or `zh_core_web_sm` , please remove these two from yml file and install separately in your env via

```
conda activate llm-26 # or conda activate llm-26-gpu
python -m spacy download en_core_web_sm
python -m spacy download zh_core_web_sm
```

After the above steps, you are ready to download our course github project.

- **Step 4: Open your jupyter notebook.**

  All your code will run on Jupyter notebook, you can activate jupyter notebook, via

  ```
  cd llm-26 # make sure you are in our course folder
  ```

  ```
  conda activate llm-26-cpu # or conda activate llm-26-gpu
  ```

  ```
  jupyter lab
  ```

  For students using Windows WSL (Ubuntu 22.04), even though Jupyter runs inside WSL (Ubuntu), you can open it directly in the Windows browser via port forwarding (usually automatic). In WSL Ubuntu, start Jupyter like this:

  ```
  jupyter lab --no-browser --ip=127.0.0.1 --port=8888
  ```

  It will print a URL like: `http://127.0.0.1:8888/lab?token=...`

  Now on Windows, open your browser and go to: `http://localhost:8888/lab` Paste the token if it asks. On WSL2, Windows automatically forwards localhost:8888 to the WSL instance in most setups. If localhost:8888 doesn't work, in WSL run: `hostname -I` , suppose it prints something like 172.27.123.45 ..., then open in Windows: `http://172.27.123.45:8888/lab`

- **Load all necessary packages**

  At this point, your env is ready to use. We import all necessary packages here.

In [1]:
```
import re
import os
import time
import math

import shutil
import socket
import platform
import requests
import datetime
import tarfile

import spacy
import torch
import numpy as np

from transformers import AutoTokenizer
from transformers import Qwen2Tokenizer, Qwen2TokenizerFast

import tiktoken
from tokenizers import normalizers
from tokenizers.normalizers import NFD, StripAccents
from tokenizers import decoders

from transformers import AutoTokenizer
from tokenizers import Tokenizer
from tokenizers import normalizers
from tokenizers.models import WordPiece
from tokenizers.normalizers import NFD, Lowercase, StripAccents
from tokenizers.pre_tokenizers import Whitespace
from tokenizers.processors import TemplateProcessing

from datasets import load_dataset
import matplotlib.pyplot as plt

jnk_start_time = time.time()
```

0.2 Checking your device

- After install conda and your env, you can check whether these packages are installed in the right way.

```
python -c "import torch; print('torch', torch.__version__); print('cud
a?', torch.cuda.is_available()); print('mps?', getattr(torch.backends,'mp
s',None) is not None and torch.backends.mps.is_available()); print('cuda
device:', torch.cuda.get_device_name(0) if torch.cuda.is_available() else
'no gpu'); print('using:', 'cuda' if torch.cuda.is_available() else ('mp
s' if (getattr(torch.backends,'mps',None) is not None and torch.backends.
mps.is_available()) else 'cpu'))"
```

In [2]:
```
!python -c "import torch; print('torch', torch.__version__); print
('cuda?', torch.cuda.is_available()); print('mps?', getattr(torch.b
ackends,'mps',None) is not None and torch.backends.mps.is_available
()); print('cuda device:', torch.cuda.get_device_name(0) if torch.c
uda.is_available() else 'no gpu'); print('using:', 'cuda' if torch.
cuda.is_available() else ('mps' if (getattr(torch.backends,'mps',No
ne) is not None and torch.backends.mps.is_available()) else 'cp
u'))"
```

```
torch 2.9.1
cuda? False
mps? True
cuda device: no gpu
using: mps
```

- The following code provides a more symmetric way to perform the same checks. In our lectures, some datasets are very large, so it's a good idea to make sure you have enough disk space. You can check your disk, memory, and CPU information using the code below.

In [3]:
```
def detect_torch_device(verbose: bool = True) -> str:
    """
    Returns one of: 'cuda', 'mps', 'cpu'
    Priority: CUDA GPU > Apple MPS > CPU
    """
    has_cuda = torch.cuda.is_available()
    has_mps = getattr(torch.backends, "mps", None) is not None and
torch.backends.mps.is_available()

    if has_cuda:
        device = "cuda"
    elif has_mps:
        device = "mps"
    else:
        device = "cpu"

    if verbose:
        print(f"torch: {torch.__version__}")
        print(f"device: {device}")

        if has_cuda:
            print(f"cuda devices: {torch.cuda.device_count()}")
            for i in range(torch.cuda.device_count()):
                print(f"  [{i}] {torch.cuda.get_device_name(i)}")
        elif has_mps:
            print("mps available: True (Apple Metal)")
        else:
            print("cpu only")

    return device

device = detect_torch_device()
# generate 2x3 random matrix to check torch
x = torch.rand(2, 3)
x = x.to(device)
print("device:", x.device)
```

```
torch: 2.9.1
device: mps
mps available: True (Apple Metal)
device: mps:0
```

- **Exercise System Info Cell**

In [4]:
```
def bytes_to_gb(x: int) -> float:
    return x / (1024 ** 3)

def system_report(path: str = "."):
    print("=== System Report ===")
    print("OS:", platform.system(), platform.release())
    print("Platform:", platform.platform())
    now = datetime.datetime.now().astimezone()
    print("Local time:", now.strftime("%Y-%m-%d %H:%M:%S %Z%z"))
    print("Hostname  :", socket.gethostname())
    print("User      :", os.getenv("USER") or os.getenv("USERNAME")
or "unknown")

    print("\n=== OS / Python ===")
    print("OS        :", platform.system(), platform.release())
    print("Version   :", platform.version())
    print("Machine   :", platform.machine())
    print("Processor :", platform.processor() or "unknown")
    print("Python    :", platform.python_version())

    # CPU
    print("\n--- CPU ---")
    print("CPU cores (logical):", os.cpu_count())

    # Memory (best-effort, cross-platform)
    print("\n--- Memory (RAM) ---")
    try:
        import psutil  # you already have this in env
        vm = psutil.virtual_memory()
        print(f"Total: {bytes_to_gb(vm.total):.2f} GB")
        print(f"Available: {bytes_to_gb(vm.available):.2f} GB")
        print(f"Used: {bytes_to_gb(vm.used):.2f} GB ({vm.percen
t}%)")
    except Exception as e:
        print("psutil not available or failed:", e)

    # Disk
    print("\n--- Disk ---")
    total, used, free = shutil.disk_usage(path)
    print("Path checked:", os.path.abspath(path))
    print(f"Total: {bytes_to_gb(total):.2f} GB")
    print(f"Free:  {bytes_to_gb(free):.2f} GB")
    print(f"Used:  {bytes_to_gb(used):.2f} GB")

    # PyTorch device
    print("\n--- PyTorch Device ---")
    try:
        import torch
        cuda = torch.cuda.is_available()
        mps = hasattr(torch.backends, "mps") and torch.backends.mp
s.is_available()
        print("torch:", torch.__version__)
        print("CUDA available:", cuda)
        print("MPS available:", mps)
        if cuda:
            print("GPU:", torch.cuda.get_device_name(0))
        device = "cuda" if cuda else ("mps" if mps else "cpu")
        print("Suggested device:", device)
    except Exception as e:
        print("torch not available or failed:", e)
```

```
system_report(".")
```

```
=== System Report ===
OS: Darwin 25.3.0
Platform: macOS-26.3-arm64-arm-64bit
Local time: 2026-03-05 10:11:37 CST+0800
Hostname  : Baojian-MBP-M3
User      : baojianzhou

=== OS / Python ===
OS        : Darwin 25.3.0
Version   : Darwin Kernel Version 25.3.0: Wed Jan 28 20:54:22 PST 2
026; root:xnu-12377.81.4~5/RELEASE_ARM64_T6030
Machine   : arm64
Processor : arm
Python    : 3.12.9

--- CPU ---
CPU cores (logical): 12

--- Memory (RAM) ---
Total: 18.00 GB
Available: 3.45 GB
Used: 6.36 GB (80.8%)

--- Disk ---
Path checked: /Users/baojianzhou/git/llm-26/lecture-01-tokenization
Total: 926.35 GB
Free:  284.86 GB
Used:  641.50 GB

--- PyTorch Device ---
torch: 2.9.1
CUDA available: False
MPS available: True
Suggested device: mps
```

1. Explore LLMs via ollama

In this section, we'll have some fun exploring LLMs with [ollama (https://github.com/ollama/ollama)](https://github.com/ollama/ollama).

1.1 Install ollama and download LLMs

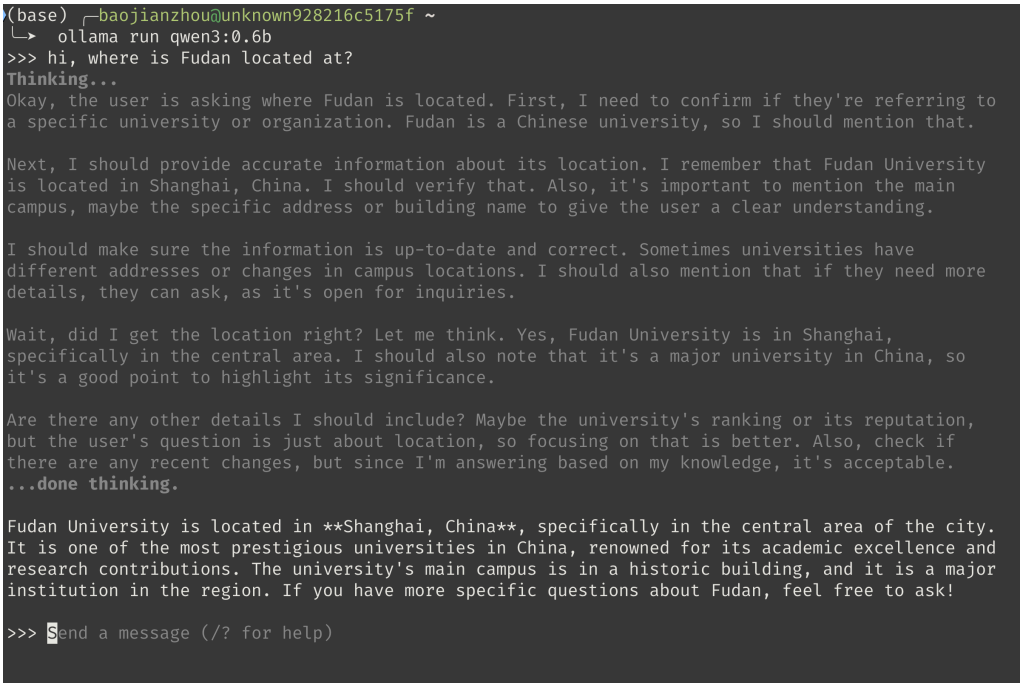
We can download some popular LLMs such as Qwen series.

- **Step 1:** Please download and install ollama via download (https://ollama.com/download).
- **Step 2:** After the installation of ollama, you can open a terminal and download qwen3:0.6b and qwen3:1.7b via:

  ```
  ollama run qwen3:0.6b
  ```

  It will automatically download the model into your local disk. It takes about 498MB disk space. Similarly, you can run a larger version `qwen3:1.7b` and it takes about 1.3GB disk space. Let us test 0.6b model

  ```
  ollama run qwen3:0.6b
  ```
- **Step 2:** Ask a question, like: `hi, where is Fudan located at?` , it gives us:



1.2 Get response for a given prompt

- **Call Ollama API**

  Next, we go a little deeper: we will call the Ollama API from Python to generate outputs given prompts. You can check whether Ollama is running by typing http://127.0.0.1:11434/ (http://127.0.0.1:11434/) in Chrome.

In [5]:
```
# Make sure bypass proxies for local services (e.g., Ollama on loca
lhost:11434)
os.environ["OLLAMA_HOST"] = "http://127.0.0.1:11434"
os.environ["NO_PROXY"] = "localhost,127.0.0.1"
os.environ["no_proxy"] = "localhost,127.0.0.1"

s = requests.Session()
# IMPORTANT <- ignore ALL proxy env vars
s.trust_env = False
# make sure ollama is running
print(s.get("http://127.0.0.1:11434/api/version", timeout=3).json
())

# !!!! make sure to import after the s.trust_env !!!!

import ollama
from ollama import chat
from ollama import ChatResponse

s = requests.Session()
s.trust_env = False  # <- ignore ALL proxy env vars

response: ChatResponse = chat(model='qwen3:0.6b', messages=[
  {
    'role': 'user',
    'content': 'Fudan University is located in which city? Answer w
ith one word.',
  },
])
print(response['message']['content'])
```

```
{'version': '0.17.5'}
Beijing
```

- **Will the same prompt always produce the same output?**

In [6]:
```
models = ["qwen3:0.6b", "qwen3:1.7b"]
prompt = "Fudan University is located in which city? Answer with on
e word."

for model in models:
    print('-' * 50)
    start_time = time.time()
    for _ in range(10):
        resp = ollama.generate(
            model = model,
            prompt = prompt
        )
        print(f"{model} with resp {_ + 1}: {resp["response"]}")
    print(f'total runtime of 10 responses of {model} is: {time.time
() - start_time:.1f} seconds')
```

```
--------------------------------------------------
qwen3:0.6b with resp 1: Beijing
qwen3:0.6b with resp 2: Hangzhou
qwen3:0.6b with resp 3: Shanghai
qwen3:0.6b with resp 4: Hangzhou
qwen3:0.6b with resp 5: Shanghai
qwen3:0.6b with resp 6: Hangzhou
qwen3:0.6b with resp 7: Shanghai
qwen3:0.6b with resp 8: Hangzhou
qwen3:0.6b with resp 9: Hangzhou
qwen3:0.6b with resp 10: Shanghai
total runtime of 10 responses of qwen3:0.6b is: 9.6 seconds
--------------------------------------------------
qwen3:1.7b with resp 1: Shenzhen
qwen3:1.7b with resp 2: Shanghai
qwen3:1.7b with resp 3: Shanghai
qwen3:1.7b with resp 4: Shanghai
qwen3:1.7b with resp 5: Shanghai
qwen3:1.7b with resp 6: Shanghai
qwen3:1.7b with resp 7: Shanghai
qwen3:1.7b with resp 8: Shanghai
qwen3:1.7b with resp 9: Fudan
qwen3:1.7b with resp 10: Shangai
total runtime of 10 responses of qwen3:1.7b is: 23.4 seconds
```

- **Some key observations**:
  - The smaller model, **Qwen3:0.6B**, tends to produce lower-quality answers, while the larger model, **Qwen3:1.7B**, generally produces higher-quality answers.
  - The **responses are somewhat random**: running the same prompt multiple times may yield different outputs. There are ways to make the output deterministic. For example, the code below sets decoding options so that the model always selects the highest-probability token at each step.

```
resp = ollama.generate(
model=model,
prompt=prompt,
options={
    "temperature": 0.0,
    "top_p": 1.0,
    "top_k": 0,
    # optional:
    "num_predict": 32,
},
)
print(resp["response"])
```

Let us generate some response that are not one word but a sequence of words.

- **Get a paragraph of response with qwen3:1.7b**

In [7]:
```
model = "qwen3:1.7b"
prompt = "I am an undergraduate student, please explain LLMs in thr
ee sentences."
resp = ollama.generate(
            model=model,
            prompt=prompt
        )
print(f"Prompt: {prompt} \nResp: {resp["response"]}")
```

```
Prompt: I am an undergraduate student, please explain LLMs in three
sentences.
Resp: Large Language Models (LLMs) are advanced AI systems trained
on massive text datasets to understand and generate human-like lang
uage. They analyze patterns in text to predict words, answer questi
ons, and assist with creative tasks like writing or coding. LLMs ar
e widely used in fields such as education, research, and business t
o enhance productivity and innovation.
```

1.3 Get response probability from LLMs

- **Get token probability (Observe: single word could be tokenized into subwords.)**

In [8]:
```
model = "qwen3:0.6b" # "qwen3:1.7b"
prompt = "Fudan University is located in which city? Answer with on
e word."
num_top_tokens = 20 # number of top alternatives per generated toke
n
resp = ollama.generate(
    model = model,
    prompt = prompt,
    stream = False,
    logprobs = True,
    think = False,
    top_logprobs = num_top_tokens
)
print("response:", repr(resp["response"]))

# Each element usually corresponds to one generated token
for i, lp in enumerate(resp.get("logprobs", [])):
    tok = lp.get("token")
    logp = lp.get("logprob")
    p = math.exp(logp) if logp is not None else None
    print(f"{i:02d} token={tok!r:>16} logp={logp: .4f} p={p:.4f}")
```

```
response: 'Shanghai'
00 token=            'Sh' logp=-0.8801 p=0.4147
01 token=        'anghai' logp=-0.1045 p=0.9008
```

- **Get specific token probability distribution via logprobs (without thinking)**

In [9]:
```
model = "qwen3:0.6b"
prompt = "Fudan University is located in which city? Answer with on
e word."

res = ollama.generate(
    model=model,
    prompt=prompt,
    logprobs=True,
    think = False,
    top_logprobs=10,
    options={"temperature": 0.0, # greedy decoding, it pick the max
imal token
             "num_predict": 20,
            "think": False # do not use thinking model.
            },
)

answer = ''
lp = res["logprobs"]
tokens = [d.get("token", "") for d in lp]
print(f'We use model: {model}')
for i in range(len(lp)):
    tok = lp[i].get("token", "")
    logp = lp[i].get("logprob", None)
    alts = lp[i].get("top_logprobs", [])
    p = math.exp(logp) if logp is not None else float("nan")
    if tok == "\n" or tok == "\n\n": # stop when answer ends (often
newline).
        break
    answer += tok
    print(f"--- top probabilities of token-{i:02d} ---")
    for a in alts[:20]:
        prob_a = math.exp(a['logprob'])
        print(f"{a['token']!r:>12}:{prob_a:.5f}")
    print(f"\nPartial Response: {answer}\n")
print(f"Final Response: {answer}")
```

```
We use model: qwen3:0.6b
--- top probabilities of token-00 ---
        'Sh':0.41104
        'Be':0.34878
        'Gu':0.07447
        'Ch':0.04332
         'F':0.02620
         'H':0.02520
         'X':0.01208
      'Hang':0.00941
         'Y':0.00885
         'J':0.00584

Partial Response: Sh

--- top probabilities of token-01 ---
    'anghai':0.89976
    'enzhen':0.04606
         'i':0.02823
       'eny':0.00730
       'and':0.00500
       'ang':0.00463
        'ij':0.00373
        'an':0.00140
        'en':0.00136
        'iz':0.00024

Partial Response: Shanghai

Final Response: Shanghai
```

- **LLMs are probabilitic distributions**

From the outputs above, you can see that the response from these LLMs may differ from run to run. In fact, LLMs are **probabilistic models**: given the same prompt (i.e., question), they may produce different responses (i.e., answers). We can describe this inference process using mathematical notation. Let $p_\theta(\cdot)$ denote a trained language model. Here, you can think of p_θ as **Qwen3:0.6B**, **Qwen3:1.7B**, or any other model. Given the following prompt

> Prompt = Fudan University is located in which city? Answer with one word.

This prompt is a sequence of tokens. The model outputs a response, which is also a sequence of tokens. In other words, the model uses an inference procedure to predict the next token(s) conditioned on the prompt. In math, it models the conditional probability

$$p_\theta\left(w_{n+1} \mid w_1, w_2, \ldots, w_n\right),$$

where

- **Prompt**$[w_1, w_2, \ldots, w_n]$ =[Fudan University is located in which city? Answer with one word.],
- **Response** = $[w_{n+1}]$ (in this simplified one-word setting).

The model then outputs the most likely token w_{n+1} (or samples from the distribution) using its own inference algorithm. We will discuss inference strategies in later lectures. By the definition of conditional probability, we have

$$p_\theta\left(w_{n+1} \mid w_1, w_2, \ldots, w_n\right) = \frac{p_\theta\left(w_1, w_2, \ldots, w_n, w_{n+1}\right)}{p_\theta\left(w_1, w_2, \ldots, w_n\right)}.$$

The key takeaway is: **if we can learn a model that assigns probabilities to sequences of arbitrary length**, i.e., $p_\theta(w_1, w_2, \ldots, w_k)$ for any positive integer k, then conditional probabilities follow naturally from these joint probabilities. Let us do one more example to see when the response is more than one word.

- **Get a sequence of token probabilities from Qwen3:1.7b**

In [10]:
```
model = "qwen3:1.7b"
prompt = "Fudan University is located in which city? Answer with on
e word."

res = ollama.generate(
    model=model,
    prompt=prompt,
    logprobs=True,
    think = False,
    top_logprobs=10,
    options={"temperature": 0.0, # greedy decoding, it pick the max
imal token
             "num_predict": 20,
            "think": False # do not use thinking model.
            },
)
print(res["response"])
```

```
Fudan University is located in **Shanghai**.
```

- If we do not use thinking mode, it gives the above response. However, we can still see how the token *Shanghai* is chosen during the inference stage. You will see these subword tokens have high probabilities.

In [11]:
```
model = "qwen3:1.7b"
prompt = "Fudan University is located in which city? Answer with on
e word."
print(f'We use model: {model}')

res = ollama.generate(
    model=model,
    prompt=prompt,
    logprobs=True,
    think = False, # Do not use the thinking model.
    top_logprobs=10,
    options={"temperature": 0.0, # greedy decoding, it pick the max
imal token
             "num_predict": 20,
            "think": False # do not use thinking model.
            },
)

answer = ''
lp = res["logprobs"]
tokens = [d.get("token", "") for d in lp]
for i in range(len(lp)):
    tok = lp[i].get("token", "")
    logp = lp[i].get("logprob", None)
    alts = lp[i].get("top_logprobs", [])
    p = math.exp(logp) if logp is not None else float("nan")
    # stop when answer ends (often newline).
    if tok == "\n" or tok == "\n\n":
        break
    answer += tok
    print(f"--- top probabilities of token-{i:02d} ---")
    for a in alts[:5]:
        prob_a = math.exp(a['logprob'])
        print(f"{a['token']!r:>5}:{prob_a:.5f}", end="")
    print(f"\nPartial Response: {answer}\n")
print(f"Final Response: {answer}")
```

```
We use model: qwen3:1.7b
--- top probabilities of token-00 ---
  'F':0.99934 'Sh':0.00026 'Fu':0.00018 'An':0.00017  'P':0.00001
Partial Response: F

--- top probabilities of token-01 ---
 'ud':1.00000 'uj':0.00000 'uz':0.00000'uding':0.00000'udu':0.00000
Partial Response: Fud

--- top probabilities of token-02 ---
 'an':1.00000'ans':0.00000 'am':0.00000 'un':0.00000 'aн':0.00000
Partial Response: Fudan

--- top probabilities of token-03 ---
' University':1.00000'University':0.00000' Univers':0.00000' Univ':
0.00000' Universe':0.00000
Partial Response: Fudan University

--- top probabilities of token-04 ---
' is':1.00000' adalah':0.00000'是中国':0.00000  '是':0.00000' là':0.0
0000
Partial Response: Fudan University is

--- top probabilities of token-05 ---
' located':1.00000' Located':0.00000 '位于':0.00000' situated':0.000
00'located':0.00000
Partial Response: Fudan University is located

--- top probabilities of token-06 ---
' in':1.00000  '在':0.00000 ' в':0.00000' 0.00000:'في' on':0.00000
Partial Response: Fudan University is located in

--- top probabilities of token-07 ---
' **':1.00000' Shanghai':0.00000' __':0.00000' the':0.00000 ' *':0.
00000
Partial Response: Fudan University is located in **

--- top probabilities of token-08 ---
 'Sh':0.80846  'W':0.07002  'P':0.05182  'F':0.03410  'N':0.01203
Partial Response: Fudan University is located in **Sh

--- top probabilities of token-09 ---
'anghai':0.99915'ang':0.00079'enzhen':0.00003 'en':0.00001'eng':0.0
0001
Partial Response: Fudan University is located in **Shanghai

--- top probabilities of token-10 ---
 '**':1.00000'**,':0.00000'***':0.00000'**:':0.00000' City':0.00000
Partial Response: Fudan University is located in **Shanghai**

--- top probabilities of token-11 ---
  '.':0.99999'.\n\n':0.00001'.\n':0.00000  '. ':0.00000 '.<':0.00000
Partial Response: Fudan University is located in **Shanghai**.

Final Response: Fudan University is located in **Shanghai**.
```

Above response process is essentially next-token prediction task as qwen3:1.7b is an AR model!

2. Basics for Python and spaCy

- **Python**: We will use Python-3.12 in our course.
- **spaCy**: As introduced in https://github.com/explosion/spaCy (https://github.com/explosion/spaCy), spaCy (https://spacy.io/) is a library for advanced Natural Language Processing in Python and Cython. It's built on the very latest research, and was designed from day one to be used in real products. We will use it to demonostrate how to do text tokenization.
- **nltk tool (outdated, will not cover in our lectures)**: In our previous courses, we usually introduce https://github.com/nltk/nltk (https://github.com/nltk/nltk) tool for tokenization stuff. But we will not cover this part as these tools are largely iirelvant and outdated.

In [12]:
```
# packages needed in this section
import re
import os
import spacy
import numpy as np
import matplotlib.pyplot as plt
```

2.1 Unicode for text data

- **Encoding Strings to Bytes**

  Use the *.encode()* method on a string object to convert it into a bytes object using UTF-8.

In [13]:
```
my_string = "Hello, World! 你好，世界！"
encoded_bytes = my_string.encode('utf-8') # Encode the string to UT
F-8 bytes
print(encoded_bytes)
```

```
b'Hello, World! \xe4\xbd\xa0\xe5\xa5\xbd\xef\xbc\x8c\xe4\xb8\x96\xe
7\x95\x8c\xef\xbc\x81'
```

- **Decoding Bytes to Strings**

  Use the *.decode()* method on a bytes object to convert it back into a string, specifying the encoding used to generate the bytes.

In [14]:
```
encoded_bytes = b'Hello, World! \xe4\xbd\xa0\xe5\xa5\xbd\xef\xbc\x8
c\xe4\xb8\x96\xe7\x95\x8c\xef\xbc\x81'
decoded_string = encoded_bytes.decode('utf-8') # Decode the bytes b
ack to a string
print(decoded_string)
```

```
Hello, World! 你好，世界！
```

- **Write and Read utf-8 text into/from files**

In [15]:
```
with open('unicode_file.txt', 'w', encoding='utf-8') as f:
    f.write("Some text with a special character: é, 你好，世界！ ")

with open('unicode_file.txt', 'r', encoding='utf-8') as f:
    content = f.read()
    print(content)
```

```
Some text with a special character: é, 你好，世界！
```

- **ASCII (American Standard Code for Information Interchange)**.

  ASCII represented each character with a single byte in the unicode. UTF-8 is a variable-length encoding format. In Python 3, UTF-8 is the default encoding for source files, strings, and many operations

In [16]:
```
import pandas as pd

rows = []
for dec in range(128):
    ch = chr(dec)
    ch_disp = repr(ch)[1:-1] if (dec < 32 or dec == 127) else ch
    rows.append({"Ch": ch_disp, "Hex": f"{dec:02X}", "Dec": dec})
df = pd.DataFrame(rows)

nblocks = 8
chunk = (len(df) + nblocks - 1) // nblocks  # ceil

blocks = []
for i in range(nblocks):
    part = df.iloc[i * chunk : (i + 1) * chunk].reset_index(drop=Tr
ue)
    part.columns = pd.MultiIndex.from_product([[f"Block {i+1}"], pa
rt.columns])
    blocks.append(part)

df3 = pd.concat(blocks, axis=1)
sty = (
    df3.style
      .set_properties(**{"text-align": "center"})
      .set_table_styles([
          {"selector": "th.col_heading.level0", "props": [("text-al
ign", "center")]},
          {"selector": "th, td", "props": [("border", "1px solid #b
bb"), ("padding", "2px 6px")]},
          {"selector": "table", "props": [("border-collapse", "coll
apse")]},
      ])
)
sty
```

Out[16]:

	Block 1			Block 2			Block 3			Block 4			Block 5			B
	Ch	Hex	Dec	Ch	Hex	Dec	Ch	Hex	Dec	Ch	Hex	Dec	Ch	Hex	Dec	Ch
0	\x00	00	0	\x10	10	16		20	32	0	30	48	@	40	64	P
1	\x01	01	1	\x11	11	17	!	21	33	1	31	49	A	41	65	Q
2	\x02	02	2	\x12	12	18	"	22	34	2	32	50	B	42	66	R
3	\x03	03	3	\x13	13	19	#	23	35	3	33	51	C	43	67	S
4	\x04	04	4	\x14	14	20	$	24	36	4	34	52	D	44	68	T
5	\x05	05	5	\x15	15	21	%	25	37	5	35	53	E	45	69	U
6	\x06	06	6	\x16	16	22	&	26	38	6	36	54	F	46	70	V
7	\x07	07	7	\x17	17	23	'	27	39	7	37	55	G	47	71	W
8	\x08	08	8	\x18	18	24	(	28	40	8	38	56	H	48	72	X
9	\t	09	9	\x19	19	25	)	29	41	9	39	57	I	49	73	Y
10	\n	0A	10	\x1a	1A	26	*	2A	42	:	3A	58	J	4A	74	Z
11	\x0b	0B	11	\x1b	1B	27	+	2B	43	;	3B	59	K	4B	75	[
12	\x0c	0C	12	\x1c	1C	28	,	2C	44	<	3C	60	L	4C	76	\
13	\r	0D	13	\x1d	1D	29	-	2D	45	=	3D	61	M	4D	77	]
14	\x0e	0E	14	\x1e	1E	30	.	2E	46	>	3E	62	N	4E	78	^
15	\x0f	0F	15	\x1f	1F	31	/	2F	47	?	3F	63	O	4F	79	_

In [17]:
```
my_string = "Hello, World! अनुच्छेद, 你好, 世界! "

encodings = ["utf-8", "utf-16-le", "gb18030"] # diiferent encoding
method
print("\nEncoded bytes (hex):")
for enc in encodings:
    b = my_string.encode(enc)
    print(f"{enc:9s}  {b.hex(' ')}")

# 3) Per-character UTF-8 bytes (useful for teaching)
print("\nPer-Char      Code-Point   UTF-8-Bytes")
for ch in my_string:
    b = ch.encode("utf-8")
    print(f"{repr(ch):>6}    ->    U+{ord(ch):04X}    ->    {b.hex
(' ')}")
```

```
Encoded bytes (hex):
utf-8      48 65 6c 6c 6f 2c 20 57 6f 72 6c 64 21 20 e0 a4 85 e0 a4
a8 e0 a5 81 e0 a4 9a e0 a5 8d e0 a4 9b e0 a5 87 e0 a4 a6 2c 20 e4 b
d a0 e5 a5 bd ef bc 8c e4 b8 96 e7 95 8c ef bc 81
utf-16-le  48 00 65 00 6c 00 6c 00 6f 00 2c 00 20 00 57 00 6f 00 72
00 6c 00 64 00 21 00 20 00 05 09 28 09 41 09 1a 09 4d 09 1b 09 47 0
9 26 09 2c 00 20 00 60 4f 7d 59 0c ff 16 4e 4c 75 01 ff
gb18030    48 65 6c 6c 6f 2c 20 57 6f 72 6c 64 21 20 81 31 cd 33 81
31 d0 38 81 31 d3 33 81 31 cf 34 81 31 d4 35 81 31 cf 35 81 31 d3 3
9 81 31 d0 36 2c 20 c4 e3 ba c3 a3 ac ca c0 bd e7 a3 a1

Per-Char      Code-Point   UTF-8-Bytes
   'H'    ->    U+0048    ->    48
   'e'    ->    U+0065    ->    65
   'l'    ->    U+006C    ->    6c
   'l'    ->    U+006C    ->    6c
   'o'    ->    U+006F    ->    6f
   ','    ->    U+002C    ->    2c
   ' '    ->    U+0020    ->    20
   'W'    ->    U+0057    ->    57
   'o'    ->    U+006F    ->    6f
   'r'    ->    U+0072    ->    72
   'l'    ->    U+006C    ->    6c
   'd'    ->    U+0064    ->    64
   '!'    ->    U+0021    ->    21
   ' '    ->    U+0020    ->    20
   'अ'    ->    U+0905    ->    e0 a4 85
   'न'    ->    U+0928    ->    e0 a4 a8
   'ु'    ->    U+0941    ->    e0 a5 81
   'च'    ->    U+091A    ->    e0 a4 9a
   'ृ'    ->    U+094D    ->    e0 a5 8d
   'छ'    ->    U+091B    ->    e0 a4 9b
   'े'    ->    U+0947    ->    e0 a5 87
   'द'    ->    U+0926    ->    e0 a4 a6
   ','    ->    U+002C    ->    2c
   ' '    ->    U+0020    ->    20
   '你'    ->    U+4F60    ->    e4 bd a0
   '好'    ->    U+597D    ->    e5 a5 bd
   ', '    ->    U+FF0C    ->    ef bc 8c
   '世'    ->    U+4E16    ->    e4 b8 96
   '界'    ->    U+754C    ->    e7 95 8c
   '! '    ->    U+FF01    ->    ef bc 81
```

2.2 Python basics: regular expression

- **Disjunction []**

In [18]:
```
# Task: Find woodchuck or Woodchuck : Disjunction

test_str = "This string contains Woodchuck and woodchuck."

result=re.search(pattern="[wW]oodchuck", string=test_str)
print("Matched" if result is not None else "Not found")
result=re.search(pattern=r"[wW]ooodchuck", string=test_str)
print("Matched" if result is not None else "Not found")
```

```
Matched
Not found
```

In [19]:
```
# Find the word "woodchuck" in the following test string

test_str = "interesting links to woodchucks ! and lemurs!"
result = re.search(pattern="woodchuck", string=test_str)
print("Matched" if result is not None else "Not found")
```

```
Matched
```

In [20]:
```
# Find !, it follows the same way:

test_str = "interesting links to woodchucks ! and lemurs!"
result = re.search(pattern="!", string=test_str)
print("Matched" if result is not None else "Not found")
result = re.search(pattern="!!", string=test_str)
print("Matched" if result is not None else "Not found")
assert re.search(pattern="!!", string=test_str) == None # match not
hing
```

```
Matched
Not found
```

- **Disjunction any digit [0-9]**

In [21]:
```
# Find any single digit in a string.

result=re.search(pattern=r"[0123456789]", string="plenty of 7 to
5")
print("Matched" if result is not None else "Not found", result)
result=re.search(pattern=r"[0-9]", string="plenty of 7 to 5")
print("Matched" if result is not None else "Not found", result)
```

```
Matched <re.Match object; span=(10, 11), match='7'>
Matched <re.Match object; span=(10, 11), match='7'>
```

- **Negation:[^**

In [20]:
```
# Negation: If the caret ^ is the first symbol after [,
# the resulting pattern is negated. For example, the pattern
# [^a] matches any single character (including special characters)
except a.

# -- not an upper case letter
print(re.search(pattern=r"[^A-Z]", string="Oyfn pripetchik"))
# -- neither 'S' nor 's'
print(re.search(pattern=r"[^Ss]", string="I have no exquisite reaso
n for't"))
# -- not a period
print(re.search(pattern=r"[^.]", string="our resident Djinn"))
# -- either 'e' or '^'
print(re.search(pattern=r"[e^]", string="look up ^ now"))
# -- the pattern ‘a^b’
print(re.search(pattern=r'a\^b', string=r'look up a^b now'))
```

```
<re.Match object; span=(1, 2), match='y'>
<re.Match object; span=(0, 1), match='I'>
<re.Match object; span=(0, 1), match='o'>
<re.Match object; span=(8, 9), match='^'>
<re.Match object; span=(8, 11), match='a^b'>
```

- **Or Operation: |**

In [22]:
```
# More disjuncations with or or operation
str1 = "Woodchucks is another name for groundhog!"
result = re.search(pattern="groundhog|woodchuck",string=str1)
print(result)
```

```
<re.Match object; span=(31, 40), match='groundhog'>
```

In [23]:
```
str1 = "Find all woodchuckk Woodchuck Groundhog groundhogxxx!"
result = re.findall(pattern="[gG]roundhog|[Ww]oodchuck",string=str
1)
print(result)
```

```
['woodchuck', 'Woodchuck', 'Groundhog', 'groundhog']
```

- **Special Operations: ?,*,+,.,$**

In [24]:
```
# Some special chars

# ?: Optional previous char
str1 = "Find all color colour colouur colouuur colouyr"
result = re.findall(pattern="colou?r",string=str1)
print(result)

# *: 0 or more of previous char
str1 = "Find all color colour colouur colouuur colouyr"
result = re.findall(pattern="colou*r",string=str1)
print(result)

# +: 1 or more of previous char
str1 = "baa baaa baaaa baaaaa"
result = re.findall(pattern="baa+",string=str1)
print(result)
# .: any char
str1 = "begin begun begun beg3n"
result = re.findall(pattern="beg.n",string=str1)
print(result)
str1 = "The end."
result = re.findall(pattern=r"\.$",string=str1)
print(result)
str1 = "The end? The end. #t"
result = re.findall(pattern=r".$",string=str1)
print(result)
```

```
['color', 'colour']
['color', 'colour', 'colouur', 'colouuur']
['baa', 'baaa', 'baaaa', 'baaaaa']
['begin', 'begun', 'begun', 'beg3n']
['.']
['t']
```

In [25]:
```
# find all "the" in a raw text use above operations.

text = "If two sequences in an alignment share a common ancestor, \
mismatches can be interpreted as point mutations and gaps as indels (that \
is, insertion or deletion mutations) introduced in one or both lineages in \
the time since they diverged from one another. In sequence alignments of \
proteins, the degree of similarity between amino acids occupying a \
particular position in the sequence can be interpreted as a rough \
measure of how conserved a particular region or sequence motif is \
among lineages. The absence of substitutions, or the presence of \
only very conservative substitutions (that is, the substitution of \
amino acids whose side chains have similar biochemical properties) in \
a particular region of the sequence, suggest [3] that this region has \
structural or functional importance. Although DNA and RNA nucleotide bases \
are more similar to each other than are amino acids, the conservation of \
base pairs can indicate a similar functional or structural role."
matches = re.findall("[^a-zA-Z][tT]he[^a-zA-Z]", text)
print(matches)
```

```
[' the ', ' the ', ' the ', ' The ', ' the ', ' the ', ' the ', ' the ']
```

In [26]:
```
# Above has spaces before or after each word. A nicer way is to do the following
matches = re.findall(r"\b[tT]he\b", text)
print(matches)
```

```
['the', 'the', 'the', 'The', 'the', 'the', 'the', 'the']
```

- **Real-world task: match English-like words [A-Za-z]+(?:'[A-Za-z]+)?**

  You can decompose this task into:
  - [A-Za-z]+: Match one or more letters (A–Z or a–z). Examples: cat, PlayStation, Media
  - (?:...): A non-capturing group (grouping for structure, but findall won't return it separately).
  - [A-Za-z]+: inside the group, Match a literal apostrophe (撇号) ' followed by one or more letters. Examples matched as a whole word: don't, I'm, teacher's
  - ?: after the group, Makes the apostrophe+letters part optional (0 or 1 time).
  - So it matches:
    - ✅ we, Valkyria, PlayStation, don't, teacher's
    - ❌ numbers (2011), hyphenated tokens (real-time), non-ASCII letters (Senjō → only Senj), punctuation, @-@, etc.

In [27]:
```
word_re = re.compile(r"[A-Za-z]+(?:'[A-Za-z]+)?")

text = """don't Senjō no Valkyria 3 : Unrecorded Chronicles ( Japan
ese : 戦場のヴァルキュリア3 ,\
lit . Valkyria of the Battlefield 3 ) , commonly referred to as Val
kyria Chronicles \
III outside Japan , is a tactical role @-@ playing video game devel
oped by Sega and \
Media.Vision for the PlayStation Portable . Released in January 201
1 in Japan , \
it is the third game in the Valkyria series . Employing the same fu
sion of tactical \
and real @-@ time gameplay as its predecessors"""

tokens = word_re.findall(text)
print(len(tokens))
print(tokens)
```

```
65
["don't", 'Senj', 'no', 'Valkyria', 'Unrecorded', 'Chronicles', 'Ja
panese', 'lit', 'Valkyria', 'of', 'the', 'Battlefield', 'commonly',
'referred', 'to', 'as', 'Valkyria', 'Chronicles', 'III', 'outside',
'Japan', 'is', 'a', 'tactical', 'role', 'playing', 'video', 'game',
'developed', 'by', 'Sega', 'and', 'Media', 'Vision', 'for', 'the',
'PlayStation', 'Portable', 'Released', 'in', 'January', 'in', 'Japa
n', 'it', 'is', 'the', 'third', 'game', 'in', 'the', 'Valkyria', 's
eries', 'Employing', 'the', 'same', 'fusion', 'of', 'tactical', 'an
d', 'real', 'time', 'gameplay', 'as', 'its', 'predecessors']
```

2.3 Tokenization using spaCy (Old way)

Usually, there are two type of tokenizations:

- **Top-down tokenization (classic)**: We define a standard and implement rules to implement that kind of tokenization.
  - word tokenization (spaCy, nltk)
  - charater tokenization (also can be done via spaCy)
- **Bottom-up tokenization (modern)**: We use simple statistics of letter sequences to break up words into subword tokens.
  - subword tokenization (modern LLMs use this type!)

Tokenization via spaCy. We assume you already installed spaCy tool. If you haven't installed yet, open your terminal and activate your llm-26 env, and then run the following to download English and Chinese LMs.

```
python -m spacy download en_core_web_sm
```

```
python -m spacy download zh_core_web_sm
```

- **Top-down (rule-based) word tokenization: Tokenization via white space**

In [28]:
```
# Use split method via the whitespace " "
text = """While the Unix command sequence just removed all the numb
ers and punctuation"""
print(text.split(" "))
```

```
['While', 'the', 'Unix', 'command', 'sequence', 'just', 'removed',
'all', 'the', 'numbers', 'and', 'punctuation']
```

In [29]:
```
# But, we have punctuations, icons, and many other small issues.

text = """Don't you love 🤗 Transformers? We sure do."""
print(text.split(" "))
```

```
["Don't", 'you', 'love', '🤗', 'Transformers?', 'We', 'sure', 'd
o.']
```

- **White space cannot tokenize Chinese,Japanese,...**

In [30]:
```
text = '姚明进入总决赛'
print(text.split(" "))
```

```
['姚明进入总决赛']
```

- **spaCy works much better: Chinese tokenization**

In [31]:
```
from spacy.lang.zh import Chinese
nlp = spacy.load("zh_core_web_sm")

text = '姚明进入总决赛'
doc = nlp(text)
print([token for token in doc])

# ? can be successfully split from Transformers
text = """Don't you love 🤗 Transformers? We sure do."""
doc = nlp(text)
print([token for token in doc])

# Chinese Character level tokenization
nlp_ch = Chinese()
text = '姚明进入总决赛'
print([*nlp_ch(text)])
```

```
[姚明, 进入, 总决赛]
[Don, 't, you, love, 🤗, Transformers, ?, We, sure, do, .]
[姚, 明, 进, 入, 总, 决, 赛]
```

- **spaCy works much better: English tokenization**

In [32]:
```
import spacy
from spacy.tokens import Doc

nlp = spacy.blank("en")

text = "Hello, world!"
chars = [c for c in text]
doc = Doc(nlp.vocab, words=chars)

print([t.text for t in doc])

# ignore white space
chars = [c for c in text if not c.isspace()]
doc = Doc(nlp.vocab, words=chars)

print([t.text for t in doc])
```

```
['H', 'e', 'l', 'l', 'o', ',', ' ', 'w', 'o', 'r', 'l', 'd', '!']
['H', 'e', 'l', 'l', 'o', ',', 'w', 'o', 'r', 'l', 'd', '!']
```

- **spaCy handles special characters**

In [33]:
```
text = """Special characters and numbers will need to be kept in pr
ices ($45.55) and dates (01/02/06);
we don’t want to segment that price into separate tokens of “45” an
d “55”. And there are URLs (https://www.stanford.edu),
Twitter hashtags (#nlproc), or email addresses (someone@cs.colorad
o.edu)."""
text = text.replace("\n", " ").strip()
nlp = spacy.load('en_core_web_sm')
doc = nlp(text)
all_tokens = [token for token in doc]
print(all_tokens)
```

```
[Special, characters, and, numbers, will, need, to, be, kept, in, p
rices, (, $, 45.55, ), and, dates, (, 01/02/06, ), ;,  , we, do,
n’t, want, to, segment, that, price, into, separate, tokens, of, “,
45, ”, and, “, 55, ”, ., And, there, are, URLs, (, https://www.stan
ford.edu, ), ,, Twitter, hashtags, (, #, nlproc, ), ,, or, email, a
ddresses, (, someone@cs.colorado.edu, ), .]
```

- **Sentence-level tokenization via spaCy**

In [34]:
```
text = """自然语言处理（英语：Natural Language Processing，缩写作 NLP）是人工智能和语言学领域的交叉学科，\
研究计算机处理、理解与生成人类语言的技术。此领域探讨如何处理及运用自然语言；自然语言处理包括多方面和步骤，\
基本有认知、理解、生成等部分。自然语言认知和理解是让电脑把输入的语言变成结构化符号与语义关系，\
然后根据目的再处理。自然语言生成系统则是把计算机数据转化为自然语言。\
自然语言处理要研制表示语言能力和语言应用的模型，建立计算框架来实现并完善语言模型，\
并根据语言模型设计各种实用系统及探讨这些系统的评测技术。"""
nlp = spacy.load("zh_core_web_sm")
doc = nlp(text)
print([sent.text for sent in doc.sents])
```

```
['自然语言处理（英语：Natural Language Processing，缩写作 NLP）是人工智能和语言学领域的交叉学科，研究计算机处理、理解与生成人类语言的技术。', '此领域探讨如何处理及运用自然语言；自然语言处理包括多方面和步骤，基本有认知、理解、生成等部分。', '自然语言认知和理解是让电脑把输入的语言变成结构化符号与语义关系，然后根据目的再处理。', '自然语言生成系统则是把计算机数据转化为自然语言。', '自然语言处理要研制表示语言能力和语言应用的模型，建立计算框架来实现并完善语言模型，并根据语言模型设计各种实用系统及探讨这些系统的评测技术。', '\u200c\u200c']
```

- Or, you can spaCy provided sentences function

In [35]:
```
# You need to install it via: python -m spacy download zh_core_web_sm
from spacy.lang.zh.examples import sentences
nlp = spacy.load("zh_core_web_sm")
doc = nlp(sentences[0])
text = """\
字节对编码是一种简单的数据压缩形式，这种方法用数据中不存的一个字节表示最常出现的连续字节数据。\
这样的替换需要重建全部原始数据。字节对编码实例：假设我们要编码数据 aaabdaaabac，字节对“aa” \
出现次数最多，所以我们用数据中没有出现的字节“Z”替换“aa”得到替换表Z <- aa 数据转变为 ZabdZabac. \
在这个数据中，字节对“Za”出现的次数最多，我们用另外一个字节“Y”来替换它（这种情况下由于所有的“Z”都将被替换，\
所以也可以用“Z”来替换“Za”），得到替换表以及数据 Z <- aa, Y <- Za, YbdYbac.
"""
doc = nlp(text)
sentences = [sent.text for sent in doc.sents]
print(sentences)
```

```
['字节对编码是一种简单的数据压缩形式，这种方法用数据中不存的一个字节表示最常出现的连续字节数据。', '这样的替换需要重建全部原始数据。', '字节对编码实例：假设我们要编码数据 aaabdaaabac，字节对“aa” 出现次数最多，所以我们用数据中没有出现的字节“Z”替换“aa”得到替换表Z <- aa 数据转变为 ZabdZabac.', '在这个数据中，字节对“Za”出现的次数最多，我们用另外一个字节“Y”来替换它（这种情况下由于所有的“Z”都将被替换，所以也可以用“Z”来替换“Za”），得到替换表以及数据 Z <- aa, Y <- Za, YbdYbac.\n']
```

In [36]:
```
# A modern and fast NLP library that includes support for sentence segmentation.
# spaCy uses a statistical model to predict sentence boundaries, which can be more accurate
# than rule-based approaches for complex texts.

nlp = spacy.load("en_core_web_sm")
doc = nlp("Here is a sentence. Here is another one! And the last one.")
sentences = [sent.text for sent in doc.sents]
for ind, sent in enumerate(sentences):
    print(f"sentence-{ind}: {sent}\n")
```

```
sentence-0: Here is a sentence.

sentence-1: Here is another one!

sentence-2: And the last one.
```

- **Lemmatization (词形还原) and Stemming (词干提取)**

Lemmatization is the task of determining that two words have the same root, despite their surface differences. For some NLP situations, we also want two morphologically different forms of a word to behave similarly. For example in web search, someone may type the string woodchucks but a useful system might want to also return pages that mention woodchuck with no s.

- **Example 1**: The words am, are, and is have the shared lemma be.
- **Example 2**: The words dinner and dinners both have the lemma dinner.

Lemmatization algorithms can be complex. For this reason we sometimes make use of a simpler but cruder method, which mainly consists of chopping off words final affixes. This naive version of morphological analysis is called stemming.

In [37]:
```
import spacy
import pandas as pd

prompt = "Fudan University is located in Shanghai. are Universitie
s"
nlp = spacy.load("en_core_web_sm")
doc = nlp(prompt)

df = pd.DataFrame([{
    "text": t.text,
    "lemma": t.lemma_, # Lemma: The base form of the word.
    "pos": t.pos_, # POS: The simple UPOS part-of-speech tag.
    "tag": t.tag_, # Tag: The detailed part-of-speech tag.
    "dep": t.dep_, # Dep: Syntactic dependency, i.e. the relation b
etween tokens.
    "shape": t.shape_, # Shape: The word shape — capitalization, pu
nctuation, digits.
    "is_alpha": t.is_alpha, # is alpha: Is the token an alpha chara
cter?
    "is_stop": t.is_stop, # is stop: Is the token part of a stop li
st.
} for t in doc])
df
```

Out[37]:

	text	lemma	pos	tag	dep	shape	is_alpha	is_stop
0	Fudan	Fudan	PROPN	NNP	compound	Xxxxx	True	False
1	University	University	PROPN	NNP	nsubjpass	Xxxxx	True	False
2	is	be	AUX	VBZ	auxpass	xx	True	True
3	located	locate	VERB	VBN	ROOT	xxxx	True	False
4	in	in	ADP	IN	prep	xx	True	True
5	Shanghai	Shanghai	PROPN	NNP	pobj	Xxxxx	True	False
6	.	.	PUNCT	.	punct	.	False	False
7	are	be	AUX	VBP	ROOT	xxx	True	True
8	Universities	university	NOUN	NNS	nsubj	Xxxxx	True	False

3. Datasets Exploration

3.1 Heap's Law

Word types and word instances (tokens)

- **Word types** are the number of distinct words in a corpus; if the set of words in the vocabulary is

$$V = \{v_1, v_2, \ldots, v_{|V|}\},$$

where $|V|$ is the number of types. We also call it, the vocabulary size $|V|$.

- **Word instances**: Given a text corpus, we treat it as a sequence of tokens $[w_1, w_2, \ldots, w_T]$ after tokenization. We call T, the total number T of running words in this corpus.
- T: the number of word instances of corpus
- $|V|$: the number of word types

The larger the corpora we look at, the more word types we find, and in fact this relationship between $|V|$ and T is called **Herdan's Law** or [Heaps' Law (https://en.wikipedia.org/wiki/Heaps%27_law)](https://en.wikipedia.org/wiki/Heaps%27_law) after its discoverers (in linguistics and information retrieval respectively). Given k and β positive constants, and $0 < \beta < 1$, it has the following form

$$|V| = k \cdot T^{\beta},$$

the value of β depends on the corpus size and the genre. With English text corpora, typically $k \in [10, 100]$, and $\beta \in [0.4, 0.6]$. For the large corpora, β ranges from .67 to .75. Roughly then we can say that the vocabulary size for a text goes up significantly faster than the square root of its length in words. Let us test it! Check more on

3.2 Explore wikitext-2 dataset

- **Please note that huggingface cannot be directly used in China. An alternative way is to use [https://hf-mirror.com/ (https://hf-mirror.com/)](https://hf-mirror.com/).** If the below code does not work, please add followwing env variable in your terminal

```
export HF_ENDPOINT=https://hf-mirror.com
```

- **Explore wikitext-2 dataset**

In [38]:
```
os.environ["HF_ENDPOINT"] = "https://hf-mirror.com"
os.environ["HF_HUB_ETAG_TIMEOUT"] = "60"
os.environ["HF_HUB_DOWNLOAD_TIMEOUT"] = "60"

from datasets import load_dataset
# loads train/validation/test
wiki2_ds = load_dataset("wikitext", "wikitext-2-raw-v1")
wiki2_train = wiki2_ds["train"]
```

In [39]:
```
summary = pd.DataFrame([
    {
        "split": name,
        "num_rows": d.num_rows,
        "num_columns": d.num_columns,
        "columns": ", ".join(d.column_names),
    }
    for name, d in wiki2_ds.items()
])
summary
```

Out[39]:

	split	num_rows	num_columns	columns
0	test	4358	1	text
1	train	36718	1	text
2	validation	3760	1	text

In [40]:
```
preview = []
for name, d in wiki2_ds.items():
    for i in range(5):
        preview.append({"split": name, "idx": i, **d[i]})

pd.DataFrame(preview)
```

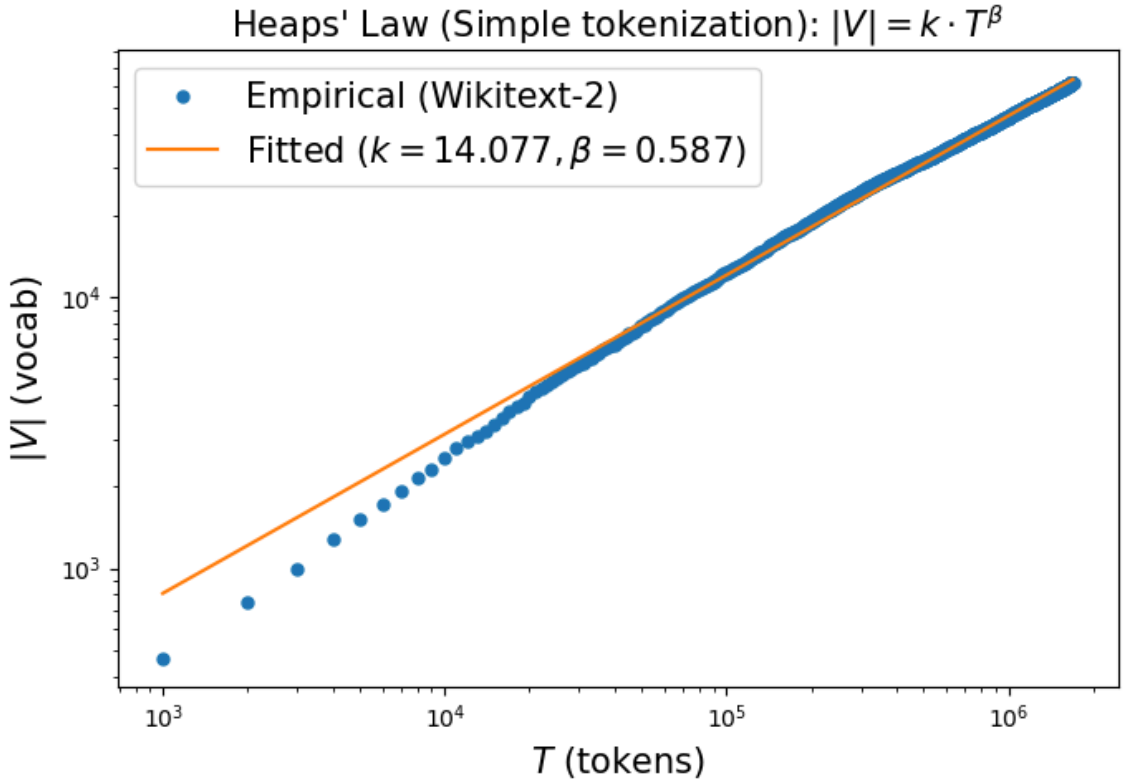
Out[40]:

	split	idx	text
0	test	0	
1	test	1	= Robert Boulter = \n
2	test	2	
3	test	3	Robert Boulter is an English film , televisio...
4	test	4	In 2006 , Boulter starred alongside Whishaw i...
5	train	0	
6	train	1	= Valkyria Chronicles III = \n
7	train	2	
8	train	3	Senjō no Valkyria 3 : Unrecorded Chronicles (...
9	train	4	The game began development in 2010 , carrying...
10	validation	0	
11	validation	1	= Homarus gammarus = \n
12	validation	2	
13	validation	3	Homarus gammarus , known as the European lobs...
14	validation	4	

- **Simple tokenization and testing the Heap's law.**

In [41]:
```
import re
import numpy as np
import matplotlib.pyplot as plt

start_time = time.time()
wiki2_train = wiki2_ds["train"]
word_re = re.compile(r"[A-Za-z]+(?:'[A-Za-z]+)?") # simple word tok
enizer

def heaps_curve(dataset, step=1000):
    V = set()
    N = 0
    Ns, Vs = [], []
    for ex in dataset:
        text = ex["text"]
        if not text or text.strip() == "": # empty word -> continue
            continue
        words = word_re.findall(text.lower()) # make all words to l
ow cases
        for w in words:
            N += 1
            V.add(w)
            if N % step == 0:
                Ns.append(N)
                Vs.append(len(V))
    return np.array(Ns), np.array(Vs)

Ns, Vs = heaps_curve(wiki2_train, step=1000)

# Fit log |V| = log k + beta log N  -> linear regression on logs
logN = np.log(Ns)
logV = np.log(Vs)
beta, logk = np.polyfit(logN, logV, 1)
k = np.exp(logk)

print(f"Fitted Heaps' law: |V| ≈ {k:.2f} * T^{beta:.3f}")
print(f"Total runtime: {time.time() - start_time:.3f} seconds")

# Plot (log-log)
plt.figure(figsize=(7, 5))
plt.loglog(Ns, Vs, marker='o', linestyle='none', markersize=5, labe
l="Empirical (Wikitext-2)")

# fitted line
Ns_line = np.linspace(Ns[0], Ns[-1], 200)
Vs_line = k * (Ns_line ** beta)
plt.loglog(Ns_line, Vs_line, label=fr"Fitted ($k={k:.3f},\beta=${be
ta:.3f})")

plt.xlabel("$T$ (tokens)", fontsize = 15)
plt.ylabel("$|V|$ (vocab)", fontsize = 15)
plt.title(r"Heaps' Law (Simple tokenization): $|V|=k \cdot T^{\bet
a}$", fontsize = 15)
plt.legend(fontsize = 15)
plt.tight_layout()
plt.show()
```

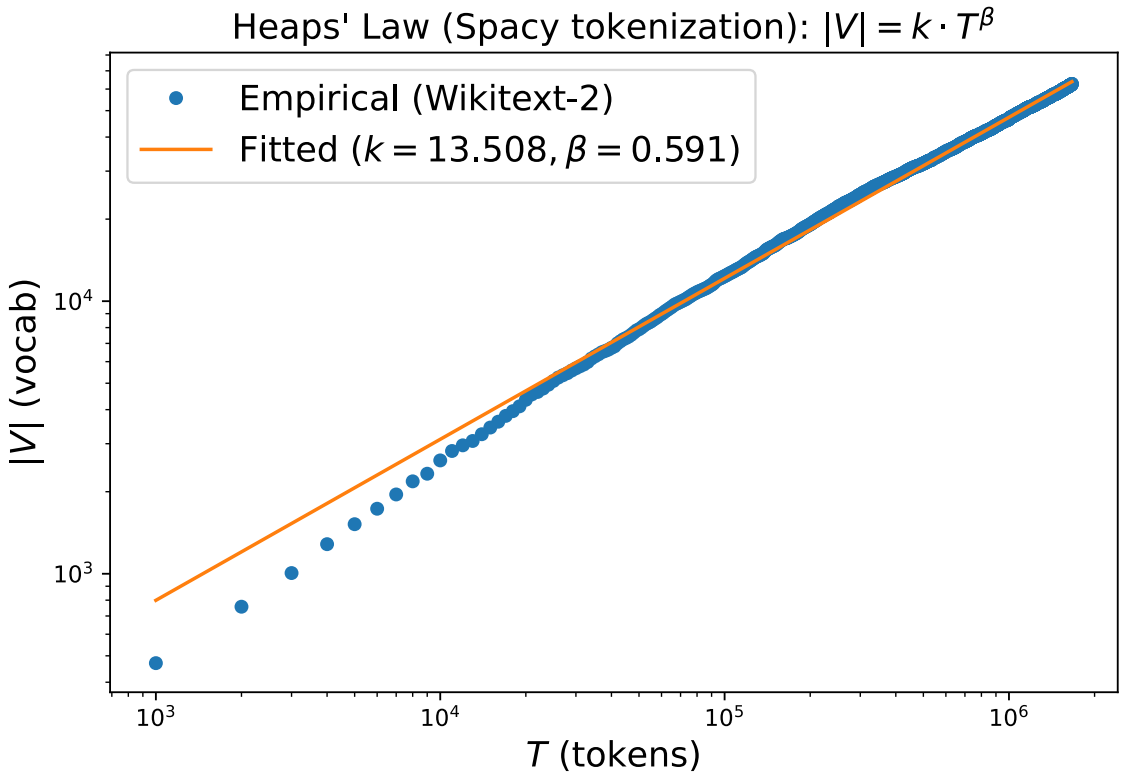
```
Fitted Heaps' law: |V| ≈ 14.08 * T^0.587
Total runtime: 0.596 seconds
```



- **Use spaCy tokenization and test Heap's Law**

In [78]:
```
start_time = time.time()

nlp = spacy.load(
    "en_core_web_sm",
    disable=["tagger","parser","ner","lemmatizer"]
)
wiki2_train = wiki2_ds["train"]

def heaps_curve_spacy(dataset, step=10_000, batch_size=256):
    V = set()
    N = 0
    Ns, Vs = [], []

    texts = (ex["text"] for ex in dataset if ex["text"] and ex["tex
t"].strip())
    for doc in nlp.pipe(texts, batch_size=batch_size):
        for tok in doc:
            if tok.is_alpha: # choose your definition of "word"
                w = tok.text.lower()
                N += 1
                V.add(w)
                if N % step == 0:
                    Ns.append(N)
                    Vs.append(len(V))
    return np.array(Ns), np.array(Vs)

Ns, Vs = heaps_curve_spacy(wiki2_train, step=1000)

# Fit log |V| = log k + beta log N
logN = np.log(Ns)
logV = np.log(Vs)
beta, logk = np.polyfit(logN, logV, 1)
k = np.exp(logk)

print(f"Fitted Heaps' law (spaCy): |V| ≈ {k:.2f} * T^{beta:.3f}")
print(f"Total runtime: {time.time() - start_time:.3f} seconds")
# Plot (log-log)
plt.figure(figsize=(7, 5))
plt.loglog(Ns, Vs, marker='o', linestyle='none', markersize=5, labe
l="Empirical (Wikitext-2)")

# fitted line
Ns_line = np.linspace(Ns[0], Ns[-1], 200)
Vs_line = k * (Ns_line ** beta)
plt.loglog(Ns_line, Vs_line, label=fr"Fitted ($k={k:.3f},\beta=${be
ta:.3f})")

plt.xlabel("$T$ (tokens)", fontsize = 15)
plt.ylabel("$|V|$ (vocab)", fontsize = 15)
plt.title(r"Heaps' Law (Spacy tokenization): $|V|=k\cdot T^{\beta}
$", fontsize = 15)
plt.legend(fontsize = 15)
plt.tight_layout()
plt.show()
```

```
Fitted Heaps' law (spaCy): |V| ≈ 13.51 * T^0.591
Total runtime: 85.812 seconds
```



3.3 Explore wikitext-103 dataset

We can download a larger dataset from the [hf-mirror (https://hf-mirror.com/)](https://hf-mirror.com/):

```
export HF_HOME=$HOME/.cache/huggingface

huggingface-cli download Salesforce/wikitext --repo-type dataset --resume
-download --include "wikitext-103-raw-v1/*.parquet"
```

- **Explore wikitext-103**

In [79]:
```
from datasets import load_dataset
wiki103_ds = load_dataset("wikitext", "wikitext-103-raw-v1")  # wil
l hit cache if present
```

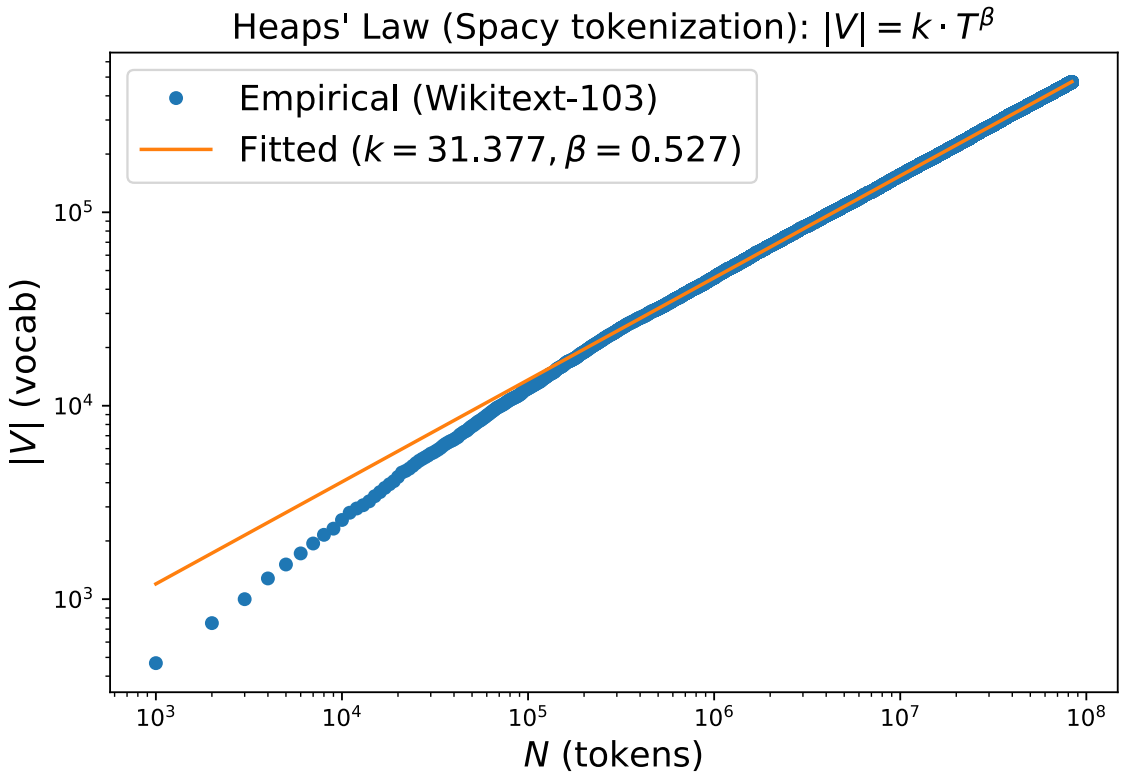
In [80]:
```
summary = pd.DataFrame([
    {
        "split": name,
        "num_rows": d.num_rows,
        "num_columns": d.num_columns,
        "columns": ", ".join(d.column_names),
    }
    for name, d in wiki103_ds.items()
])
summary
```

Out[80]:

	split	num_rows	num_columns	columns
0	test	4358	1	text
1	train	1801350	1	text
2	validation	3760	1	text

In [81]:
```
start_time = time.time()
wiki103_train = wiki103_ds["train"]
word_re = re.compile(r"[A-Za-z]+(?:'[A-Za-z]+)?")

def heaps_curve(dataset, step=1000):
    V = set()
    N = 0
    Ns, Vs = [], []
    for ex in dataset:
        text = ex["text"]
        if not text or text.strip() == "": # empty word -> continue
            continue
        words = word_re.findall(text.lower()) # make all words to l
ow cases
        for w in words:
            N += 1
            V.add(w)
            if N % step == 0:
                Ns.append(N)
                Vs.append(len(V))
    return np.array(Ns), np.array(Vs)

Ns, Vs = heaps_curve(wiki103_train, step=1000)

# Fit log |V| = log k + beta log N  -> linear regression on logs
logN = np.log(Ns)
logV = np.log(Vs)
beta, logk = np.polyfit(logN, logV, 1)
k = np.exp(logk)

print(f"Fitted Heaps' law (spaCy): |V| ≈ {k:.2f} * T^{beta:.3f}")
print(f"Total runtime: {time.time() - start_time:.3f} seconds")
# Plot (log-log)
plt.figure(figsize=(7, 5))
plt.loglog(Ns, Vs, marker='o', linestyle='none', markersize=5, labe
l="Empirical (Wikitext-103)")

# fitted line
Ns_line = np.linspace(Ns[0], Ns[-1], 200)
Vs_line = k * (Ns_line ** beta)
plt.loglog(Ns_line, Vs_line, label=fr"Fitted ($k={k:.3f},\beta=${be
ta:.3f})")

plt.xlabel("$N$ (tokens)", fontsize = 15)
plt.ylabel("$|V|$ (vocab)", fontsize = 15)
plt.title(r"Heaps' Law (Spacy tokenization): $|V|=k \cdot T^{\beta}
$", fontsize = 15)
plt.legend(fontsize = 15)
plt.tight_layout()
plt.show()
```

```
Fitted Heaps' law (spaCy): |V| ≈ 31.38 * T^0.527
Total runtime: 26.718 seconds
```



3.4 BookCorpus datasets (GPT-1)

BookCorpus dataset has been used in training GPT-1. See (papers/GPT1-Improving_Language_Understanding_by_Generative_Pre-Training-2018.pdf) GPT-1 paper (PDF)

- **Explore the BookCorpus datasets**

You can create *.dataset* (note it is .dataset not dataset) folder in llm-26 and download our file from url (https://pan.baidu.com/s/115KyzS6sLCQjfgx9Ihh7nw?).

In [45]:
```
import time
import tarfile

path = "../.datasets/books1.tar.gz"
start_time = time.time()
with tarfile.open(path, "r:gz") as tar:
    names = tar.getnames()
    print("num members:", len(names))
    print("\n".join(names[:20]))
print(f"total time to scan BookCorpus: {time.time() - start_time:.2
f} seconds")
```

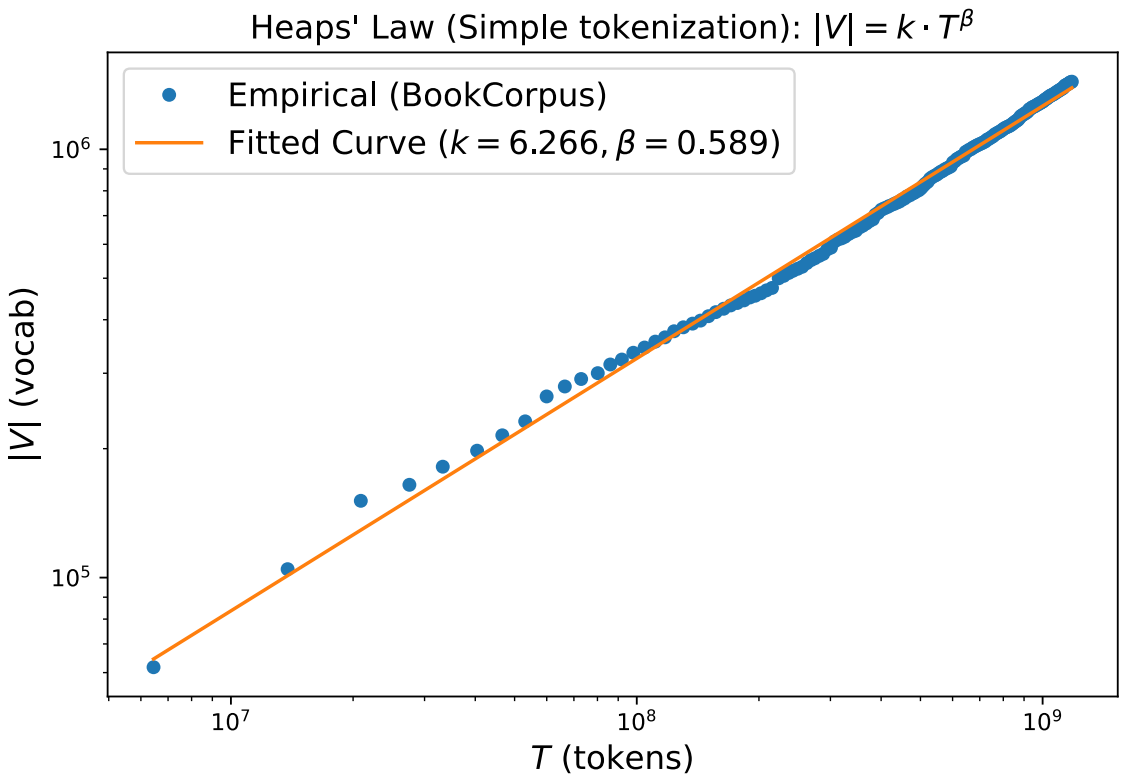
```
num members: 17871
books1
books1/epubtxt
books1/epubtxt/more-haste-the-marital-trials-of-brother-segun.epub.
txt
books1/epubtxt/the-amulet-custodian-novel-1.epub.txt
books1/epubtxt/100-seconds-to-midnight.epub.txt
books1/epubtxt/the-units.epub.txt
books1/epubtxt/gateway-to-heaven.epub.txt
books1/epubtxt/satan-the-sworn-enemy-of-mankind.epub.txt
books1/epubtxt/the-dogs-may-bark-but-the-caravan-moves-on-the-spiri
t-realm-.epub.txt
books1/epubtxt/3-book-romance-bundle-loving-the-bull-rider-cowboy-d
own-unde.epub.txt
books1/epubtxt/the-arab-states-and-the-palestine-conflict.epub.txt
books1/epubtxt/nine-ghost-stories.epub.txt
books1/epubtxt/deviations-bloodlines.epub.txt
books1/epubtxt/chronicles-of-han-behind-the-scenes-collection-of-pu
blic-blo.epub.txt
books1/epubtxt/crown-of-thorns-the-race-to-clone-jesus-christ-book-
one.epub.txt
books1/epubtxt/rifted-clouds-all-three-parts.epub.txt
books1/epubtxt/united-states-human-expedition.epub.txt
books1/epubtxt/black-shadow-detective-agency-the-shadows-up-caper.e
pub.txt
books1/epubtxt/stumps-of-mystery-stories-from-the-end-of-an-era.epu
b.txt
books1/epubtxt/paranormal-revenge.epub.txt
total time to scan BookCorpus: 20.02 seconds
```

You can check `tokenization_bookcorpus.py` to generate heaps statistics in jsonl file. (We generated them files already.)

- **Heap's Law of BookCorpus dataset**

In [125]:
```
import json
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker

def load_heaps_jsonl(log_path: str):
    Ns, Vs = [], []
    with open(log_path, "r", encoding="utf-8") as f:
        for line in f:
            r = json.loads(line)
            Ns.append(r["total_tokens"])
            Vs.append(r["vocab_size"])
    return np.array(Ns), np.array(Vs)

Ns, Vs = load_heaps_jsonl("assets/books1_wordcounts.heaps_log.jsonl")

# fit log |V| = log k + beta log N
beta, logk = np.polyfit(np.log(Ns), np.log(Vs), 1)
k = np.exp(logk)
print(f"Fit: |V| ≈ {k:.2f} * T^{beta:.3f}")


plt.figure(figsize=(7,5))

plt.loglog(Ns, Vs, marker="o", linestyle="none",
           markersize=5, label="Empirical (BookCorpus)")
Ns_line = np.linspace(Ns[0], Ns[-1], 200)
plt.loglog(Ns_line, k * (Ns_line ** beta), label=fr"Fitted Curve $(k={k:.3f},\beta=${beta:.3f})")
plt.xlabel("$T$ (tokens)", fontsize = 14)
plt.ylabel("$|V|$ (vocab)", fontsize = 14)
plt.title(r"Heaps' Law (Simple tokenization): $|V|=k \cdot T^{\beta}$", fontsize = 14)
plt.legend(fontsize = 14)
plt.tight_layout()
plt.show()
```

```
Fit: |V| ≈ 6.27 * T^0.589
```



4. LLMs tokenization (Modern Way)

In general, there are two type of tokenizations

- **Top-down tokenization**: It defines a standard and implement rules to implement that kind of tokenization. It includes, for example, word tokenization, charater tokenization, and others.
- **Bottom-up tokenization**: It uses simple statistics of letter sequences to break up words into subword tokens. Subword tokenization (modern LLMs use this type!) belongs to this type.

In case of bottom-up tokenization, there are three standard way to do modern LLM tokenization:

- **BPE tokenization:** (papers/BPE-A_New_Algorithm_for_Data_Compression-1994.pdf)[Original BPE paper], (papers/GPT2-Language_Models_are_Unsupervised_Multitask_Learners-2019.pdf)[Adopted in GPT-2]
- **Wordpiece tokenization:** (papers/Wordpiece-Japanese_and_Korean_voice_search-1994.pdf)[Original Wordpiece paper], (papers/GNMT-Google_Neural_Machine_Translation_System_Bridging_the_Gap_between_Human_and_Machine_Transla 2016.pdf)[Adopted in GNMT], (papers/BERT-Pre-training_of_Deep_Bidirectional_Transformers_for_Language_Understanding-2018.pdf)[Adopted in BERT]
- **Unigram tokenization:** (papers/Unigram-Subword_Regularization_Improving_Neural_Network_Translation_Models_with_Multiple_Subword_Candic 2018.pdf)[Original Unigram paper], (papers/ALBERT-A_Lite_BERT_for_Self-supervised_Learning_of_Language_Representations-2020.pdf)[Adopted in AlBERT], (papers/T5-model-Exploring_the_Limits_of_Transfer_Learning_with_a_Unified_Text-to-Text_Transformer-2020.pdf) [Adopted in T5]

In [90]:
```
import tiktoken
```

4.1 BPE tokenization

- **Explore trained tokenizer: [tiktoken (https://github.com/openai/tiktoken)](https://github.com/openai/tiktoken)** Any trained tokenizer has two functions:
  - **encode**: it maps running text into token ids (integers)
  - **decode**: it maps token id back to each text token (strings)
- It has been adopted from all modern LLMs including ChatGPT, GPT-series, and many others.
- tiktoken uses a byte-level BPE. (The emoji like 🤗 could be encoded as multiple UTF-8 bytes.)

4.1.1 explore tiktoken tokenizer

The [tiktoken tokenizer (https://github.com/openai/tiktoken)](https://github.com/openai/tiktoken).

In [107]:
```
# Load cl100k_tokenizer (it was used by gpt-3.5-turbo family, gpt-4, and also gpt-4-turbo

cl100k_tokenizer = tiktoken.get_encoding("cl100k_base")

text = "Don't you love 🤗 Transformers? We sure do."
token_ids = cl100k_tokenizer.encode(text)
token_strs = [cl100k_tokenizer.decode([tid]) for tid in token_ids]

print("ids:", token_ids)
print("tokens:", token_strs)
```

```
ids: [8161, 956, 499, 3021, 11410, 97, 245, 81632, 30, 1226, 2771, 656, 13]
tokens: ['Don', "'t", ' you', ' love', ' ', '', '', ' Transformers', '?', ' We', ' sure', ' do', '.']
```

- what you're seeing (' ', '', '') is normal. Those are individual bytes / partial UTF-8 pieces that don't form a valid standalone Unicode character when decoded one token at a time.
- Next, we explore o200k_base, which is the tokenizer (encoding) used by OpenAI's newer "o-series" models, especially the GPT-4o family. If you call a model like gpt-4o / gpt-4o-mini, the tokenization is o200k_base.

In [112]:
```
gpt35_tokenizer = tiktoken.get_encoding("o200k_base")

token_ids = gpt35_tokenizer.encode(text)
token_strs = [gpt35_tokenizer.decode([tid]) for tid in token_ids]

print("ids:", token_ids)
print("tokens:", token_strs)

# Count tokens by counting the length of the list returned by .enco
de().
def num_tokens_from_string(string: str, encoding_name: str) -> int:
    """Returns the number of tokens in a text string."""
    encoding = tiktoken.get_encoding(encoding_name)
    num_tokens = len(encoding.encode(string))
    return num_tokens
print("---")

text = "Don't you love 🤗 Transformers? We sure do."
print(f'\"{text}\" has been encoded into {num_tokens_from_string(te
xt, "cl100k_base")} subwords')
print(f'\"{text}\" has been encoded into {num_tokens_from_string(te
xt, "o200k_base")} subwords')
```

```
ids: [31559, 481, 3047, 93643, 245, 156303, 30, 1416, 3239, 621, 1
3]
tokens: ["Don't", ' you', ' love', ' ', '', ' Transformers', '?',
' We', ' sure', ' do', '.']
---
"Don't you love 🤗 Transformers? We sure do." has been encoded into
13 subwords
"Don't you love 🤗 Transformers? We sure do." has been encoded into
11 subwords
```

- Note the tokens generated above is slightly different from ones generated by cl100k_base.

In [114]:
```
# .decode() converts a list of token integers to a string.
encode_ids = [83, 1609, 5963, 374, 2294, 0, 11, 2025]
print(f'the decoded string is: \"{gpt35_tokenizer.decode(encode_id
s)}\"')
```

```
the decoded string is: "turesงás!,(t"
```

- Try a slightly long text.

In [119]:
```
text = """
Chapters 5 to 8 teach the basics of 🤗 Datasets and 🤗 Tokenizers b
efore diving into classic NLP tasks.\
By the end of this part, you will be able to tackle the most common
NLP problems by yourself. \
By the end of this part, you will be ready to apply 🤗 Transformers
to (almost) any machine \
learning problem! E=mc^2. f(x) = x^2+y^2, print('hello world!') bao
jianzhou. asdasfasdgasdg
"""
token_ids = gpt35_tokenizer.encode(text)
token_strs = [gpt35_tokenizer.decode([tid]) for tid in token_ids]
print(token_ids)
print(token_strs)
```

```
[198, 1205, 54524, 220, 20, 316, 220, 23, 5113, 290, 42280, 328, 93
643, 245, 415, 105003, 326, 93643, 245, 17951, 24223, 2254, 59474,
1511, 13686, 161231, 13638, 78061, 290, 1268, 328, 495, 997, 11, 48
1, 738, 413, 3741, 316, 37160, 290, 1645, 5355, 161231, 6840, 656,
6675, 13, 4534, 290, 1268, 328, 495, 997, 11, 481, 738, 413, 6977,
316, 6096, 93643, 245, 156303, 316, 350, 116393, 8, 1062, 7342, 752
4, 4792, 0, 457, 28, 24335, 61, 17, 13, 285, 4061, 8, 314, 1215, 6
1, 17, 102753, 61, 17, 11, 2123, 706, 24912, 2375, 82469, 8, 3079,
6276, 1200, 57541, 13, 472, 28922, 18865, 67, 22970, 49449, 198]
['\n', 'Ch', 'apters', ' ', '5', ' to', ' ', '8', ' teach', ' the',
' basics', ' of', ' 🤗', '🤗', ' D', 'atasets', ' and', ' 🤗', '🤗', '
Token', 'izers', ' before', ' diving', ' into', ' classic', ' NLP',
' tasks', '.By', ' the', ' end', ' of', ' this', ' part', ',', ' yo
u', ' will', ' be', ' able', ' to', ' tackle', ' the', ' most', ' c
ommon', ' NLP', ' problems', ' by', ' yourself', '.', ' By', ' th
e', ' end', ' of', ' this', ' part', ',', ' you', ' will', ' be', '
ready', ' to', ' apply', ' 🤗', '🤗', ' Transformers', ' to', ' (',
'almost', ')', ' any', ' machine', ' learning', ' problem', '!', '
E', '=', 'mc', '^', '2', '.', ' f', '(x', ')', ' =', ' x', '^',
'2', '+y', '^', '2', ',', ' print', "('", 'hello', ' world', '!’',
')', ' ba', 'oj', 'ian', 'zhou', '.', ' as', 'das', 'fas', 'd', 'ga
s', 'dg', '\n']
```

- **Limitations of general tokenizers**

Generic tokenizers can split domain strings (genes/SMILES) in semantically harmful ways; we often need domain-specific tokenization (k-mers / specialized vocabularies) or character-level modeling.
Let us consider the following example

In [121]:
```
text = """acetylsalicylic acid, aspirin, Group of substances: organ
ic Physical appearance: \
colorless needles crystals Empirical formula (Hill's system for org
anic substances): \
C9H8O4 Structural formula as text: CH3COOC6H4COOH,
the smiles format of it is CC(=O)OC1=CC=CC=C1C(O)=O."""

token_ids = gpt35_tokenizer.encode(text)
token_strs = [gpt35_tokenizer.decode([tid]) for tid in token_ids]
print(token_ids)
print(token_strs) # CC(=O)OC1=CC=CC=C1C(O)=O. should be a whole, bu
t it is tokenized into small pieces.
```

```
[86583, 3643, 25396, 3869, 459, 18655, 11, 162142, 11, 7849, 328, 4
4166, 25, 19448, 45544, 16814, 25, 3089, 2695, 94833, 70994, 15601,
138477, 20690, 350, 154150, 885, 2420, 395, 19448, 44166, 3127, 36
3, 24, 39, 23, 46, 19, 131678, 20690, 472, 2201, 25, 10049, 18, 831
0, 8695, 21, 39, 19, 8310, 45485, 11, 793, 3086, 67894, 6011, 328,
480, 382, 21547, 7, 28, 46, 8, 8695, 16, 28, 4433, 28, 4433, 93363,
16, 34, 49357, 25987, 46, 13]
['acet', 'yl', 'sal', 'icy', 'lic', ' acid', ',', ' aspirin', ',',
' Group', ' of', ' substances', ':', ' organic', ' Physical', ' app
earance', ':', ' color', 'less', ' needles', ' crystals', ' Emp',
'irical', ' formula', ' (', 'Hill', "'s", ' system', ' for', ' orga
nic', ' substances', '):', ' C', '9', 'H', '8', 'O', '4', ' Structu
ral', ' formula', ' as', ' text', ':', ' CH', '3', 'CO', 'OC', '6',
'H', '4', 'CO', 'OH', ',', ' \n', 'the', ' smiles', ' format', ' o
f', ' it', ' is', ' CC', '(', '=', 'O', ')', 'OC', '1', '=', 'CC',
'=', 'CC', '=C', '1', 'C', '(O', ')=', 'O', '.']
```

4.1.2 train a BPE tokenizer using tokenizers

- **tokenizers is fast**

  It is extremely fast (both training and tokenization), thanks to the Rust implementation. Takes less than 20 seconds to tokenize a GB of text on a server's CPU.
- **Example of training BPE using tokenizers**

  We will use wikitext-103 to train a tokenizer
  - Start with all the characters present in the training corpus as tokens.
  - Identify the most common pair of tokens and merge it into one token.
  - Repeat until the vocabulary (e.g., the number of tokens) has reached the size we want.

- **Step 1: pretraining for tokenization learner (adding whitespace)**

In [140]:
```
from datasets import load_dataset
from tokenizers import Tokenizer
from tokenizers.models import BPE
from tokenizers.trainers import BpeTrainer
from tokenizers.pre_tokenizers import Whitespace

# load wikitext-103
wiki103_ds = load_dataset("wikitext", "wikitext-103-raw-v1")  # tra
in/validation/test

# Step 1: pre-training
tokenizer = Tokenizer(BPE(unk_token="[UNK]"))
tokenizer.pre_tokenizer = Whitespace()
```

- **Step 2: training (using BPE algorithm)**

In [142]:
```
start_time = time.time()

# build tokenizer + trainer
trainer = BpeTrainer(
    vocab_size=30000,
    special_tokens=["[UNK]", "[PAD]", "[BOS]", "[EOS]", "[MASK]", "
[CLS]", "[SEP]"]
)
# iterator over ALL splits
def batch_iterator(batch_size=1000):
    for split in ["train", "validation", "test"]:
        for i in range(0, len(wiki103_ds[split]), batch_size):
            yield ds[split][i:i+batch_size]["text"]
tokenizer.train_from_iterator(batch_iterator(), trainer=trainer)
# It will take above 1 second
print(f"Total runtime of training tokenizer on wikitext-103: {time.
time() - start_time:.3f} seconds")
```

```
Total runtime of training tokenizer on wikitext-103: 0.998 seconds
```

- **Step 3: save the tokenizer**

In [158]:
```
# Step 3: save the trained tokenizer
tokenizer.save("assets/wikitext103-bpe.json")
```

In [159]:
```
tokenizer = Tokenizer.from_file("assets/wikitext103-bpe.json")
output = tokenizer.encode("Hello, y'all! How are you 😄 ?")
print(output.tokens)
print(output.ids)
```

```
['[CLS]', 'Hel', 'lo', ',', 'y', "'", 'all', '!', 'How', 'are', 'yo
u', '[UNK]', '?', '[SEP]']
[1, 3689, 1354, 18, 95, 13, 1244, 7, 4118, 1251, 2453, 0, 37, 2]
```

4.2 WordPiece tokenization

- **Step 1: normalization**

In [149]:
```
# Step 1: normalization, Normalization is, in a nutshell, a set of
operations you apply to
# a raw string to make it less random or “cleaner”. Common operatio
ns include stripping
# whitespace, removing accented characters or lowercasing all text.

from tokenizers import normalizers
from tokenizers.normalizers import NFD, StripAccents
normalizer = normalizers.Sequence([NFD(), StripAccents()])

print(normalizer.normalize_str("Héllò hôw are ü?"))
tokenizer.normalizer = normalizer
```

```
Hello how are u?
```

- **Step 2: Pre-Tokenization**

In [150]:
```
# Step 2: Pre-Tokenization, Pre-tokenization is the act of splittin
g a text into smaller
# objects that give an upper bound to what your tokens will be at t
he end of training.
# A good way to think of this is that the pre-tokenizer will split
your text into “words”
# and then, your final tokens will be parts of those words. An easy
way to pre-tokenize
# inputs is to split on spaces and punctuations, which is done by t
he pre_tokenizers.
from tokenizers.pre_tokenizers import Whitespace
pre_tokenizer = Whitespace()
pre_tokenizer.pre_tokenize_str("Hello! How are you? I'm fine, thank
you.")
```

Out[150]:
```
[('Hello', (0, 5)),
 ('!', (5, 6)),
 ('How', (7, 10)),
 ('are', (11, 14)),
 ('you', (15, 18)),
 ('?', (18, 19)),
 ('I', (20, 21)),
 ("'", (21, 22)),
 ('m', (22, 23)),
 ('fine', (24, 28)),
 (',', (28, 29)),
 ('thank', (30, 35)),
 ('you', (36, 39)),
 ('.', (39, 40))]
```

In [147]:
```
from tokenizers import pre_tokenizers
from tokenizers.pre_tokenizers import Digits
pre_tokenizer = pre_tokenizers.Sequence([Whitespace(), Digits(indiv
idual_digits=True)])
pre_tokenizer.pre_tokenize_str("Call 911!")
```

Out[147]:
```
[('Call', (0, 4)), ('9', (5, 6)), ('1', (6, 7)), ('1', (7, 8)),
('!', (8, 9))]
```

- **Step 3: Choose your model (WordPiece)**

Choose your tokenization algorithm.

- models.BPE
- models.Unigram
- models.WordLevel
- models.WordPiece

- **Step 4: Post-Processing**

Post-processing is the last step of the tokenization pipeline, to perform any additional transformation to the Encoding before it's returned, like adding potential special tokens.

In [151]:
```
from tokenizers.processors import TemplateProcessing
tokenizer.post_processor = TemplateProcessing(
    single="[CLS] $A [SEP]",
    pair="[CLS] $A [SEP] $B:1 [SEP]:1",
    special_tokens=[("[CLS]", 1), ("[SEP]", 2)],
)
```

In [152]:
```
# BERT tokenization building

start_time = time.time()

from transformers import AutoTokenizer
from tokenizers import Tokenizer
from tokenizers.models import WordPiece
bert_tokenizer = Tokenizer(WordPiece(unk_token="[UNK]"))

from tokenizers import normalizers
from tokenizers.normalizers import NFD, Lowercase, StripAccents
bert_tokenizer.normalizer = normalizers.Sequence([NFD(), Lowercase
(), StripAccents()])

from tokenizers.pre_tokenizers import Whitespace
bert_tokenizer.pre_tokenizer = Whitespace()

from tokenizers.processors import TemplateProcessing
bert_tokenizer.post_processor = TemplateProcessing(
    single="[CLS] $A [SEP]",
    pair="[CLS] $A [SEP] $B:1 [SEP]:1",
    special_tokens=[
        ("[CLS]", 1),
        ("[SEP]", 2),
    ],
)
from tokenizers.trainers import WordPieceTrainer
trainer = WordPieceTrainer(vocab_size=30522, special_tokens=["[UN
K]", "[CLS]", "[SEP]", "[PAD]", "[MASK]"])

# iterator over ALL splits
def batch_iterator(batch_size=1000):
    for split in ["train", "validation", "test"]:
        for i in range(0, len(wiki103_ds[split]), batch_size):
            yield ds[split][i:i+batch_size]["text"]
bert_tokenizer.train_from_iterator(batch_iterator(), trainer=traine
r)
# It will take above 1 second
print(f"Total runtime of training tokenizer on wikitext-103: {time.
time() - start_time:.3f} seconds")
bert_tokenizer.save("assets/wikitext103-bert.json")
```

```
Total runtime of training tokenizer on wikitext-103: 1.025 seconds
```

- **Step 5: Encoding and Decoding**

In [154]:
```
output = bert_tokenizer.encode("Hello, y'all! How are you 😄 ?")
print(output.ids)
# [1, 5993, 919, 16, 67, 11, 1772, 5, 1972, 1700, 2358, 0, 35, 2]
bert_tokenizer.decode([1, 5993, 919, 16, 67, 11, 1772, 5, 1972, 170
0, 2358, 0, 35, 2])
# "Hello , y ' all ! How are you ?"
```

```
[1, 5993, 919, 16, 67, 11, 1772, 5, 1972, 1700, 2358, 0, 35, 2]
```

Out[154]:
```
"hell ##o , y ' all ! how are you ?"
```

In [155]:
```
output = bert_tokenizer.encode("Welcome to the 🤗 Tokenizers librar
y.")
print(output.tokens)
# ["[CLS]", "welcome", "to", "the", "[UNK]", "tok", "##eni", "##ze
r", "##s", "library", ".", "[SEP]"]
bert_tokenizer.decode(output.ids)
# "welcome to the tok ##eni ##zer ##s library ."
```

```
['[CLS]', 'welcome', 'to', 'the', '[UNK]', 'tok', '##eni', '##zer',
'##s', 'library', '.', '[SEP]']
```

Out[155]:
```
'welcome to the tok ##eni ##zer ##s library .'
```

4.3 Unigram tokenization

4.4 Huggingface pretrained tokenizers

PreTrainedTokenizer, PreTrainedTokenizerBase, AutoTokenizer

- https://github.com/huggingface/transformers/blob/main/src/transformers/tokenization_utils.py (https://github.com/huggingface/transformers/blob/main/src/transformers/tokenization_utils.py)
- https://github.com/huggingface/transformers/blob/main/src/transformers/tokenization_utils_base.py (https://github.com/huggingface/transformers/blob/main/src/transformers/tokenization_utils_base.py)
- https://github.com/huggingface/transformers/blob/main/src/transformers/tokenization_utils_fast.py (https://github.com/huggingface/transformers/blob/main/src/transformers/tokenization_utils_fast.py)
- Check tokenizers at https://github.com/huggingface/transformers/blob/main/setup.py (https://github.com/huggingface/transformers/blob/main/setup.py)
- If you want to train a tokenizer by yourself, then go to: https://github.com/huggingface/tokenizers (https://github.com/huggingface/tokenizers)
- A fast BPE tokenizer is also at: https://github.com/openai/tiktoken (https://github.com/openai/tiktoken)
- An implementation of sentencepiece is at: https://github.com/google/sentencepiece (https://github.com/google/sentencepiece)
- There are 3 most common methods for tokenization: https://github.com/huggingface/tokenizers/tree/main/tokenizers/src/models (https://github.com/huggingface/tokenizers/tree/main/tokenizers/src/models)
-
  - BPE: https://aclanthology.org/P16-1162.pdf (https://aclanthology.org/P16-1162.pdf)
  - Unigram: https://arxiv.org/pdf/1804.10959 (https://arxiv.org/pdf/1804.10959)
  - WordPiece https://static.googleusercontent.com/media/research.google.com/en//pubs/archive/37842.pdf (https://static.googleusercontent.com/media/research.google.com/en//pubs/archive/37842.pdf)

In [ ]:
```
from transformers import Qwen2Tokenizer, Qwen2TokenizerFast
```

In [162]:
```
tokenizer = Qwen2TokenizerFast.from_pretrained(pretrained_model_nam
e_or_path="Qwen/Qwen-tokenizer")
text = "What is natural language processing? 🤗"
encoded = tokenizer(text, return_tensors="pt")
print("Input IDs:", encoded["input_ids"])
```

```
Input IDs: tensor([[ 3838,   374,  5810,  4128,  8692,    30, 1116
2,    97,   245]])
```

In [161]:
```
from transformers import AutoTokenizer
# Load the tokenizer
tokenizer = AutoTokenizer.from_pretrained("deepseek-ai/DeepSeek-V
3")
# Example: tokenize some text
text = "What is natural language processing? 🤗"
encoded = tokenizer(text, return_tensors="pt")
print("Input IDs:", encoded["input_ids"])
decoded = tokenizer.decode(encoded["input_ids"][0])
print("Decoded text:", decoded)
decoded = tokenizer.decode(encoded["input_ids"][0], skip_special_to
kens=True)
print("Decoded with special text:", decoded)
```

```
Input IDs: tensor([[     0,   3085,    344,   4095,   4063,   8037,
33, 112838,    248]])
Decoded text: <｜begin_of_sentence｜>What is natural language proces
sing? 🤗
Decoded with special text: What is natural language processing? 🤗
```

In [165]:
```
vocab = tokenizer.get_vocab()
print("Vocab size (attribute):", tokenizer.vocab_size)
print("Vocab size (dict length):", len(vocab))
id_to_token = {id_: tok for tok, id_ in vocab.items()}
pairs = [[i, repr(id_to_token[i])]for i in range(20)] # show first
20 tokens
print(pairs)
```

```
Vocab size (attribute): 151643
Vocab size (dict length): 151646
[[0, "'!'"], [1, '\'"\''], [2, "'#'"], [3, "'$'"], [4, "'%'"], [5,
"'&'"], [6, '"\'"'], [7, "'('"], [8, "')'"], [9, "'*'"], [10,
"'+'"], [11, "','"], [12, "'-'"], [13, "'.'"], [14, "'/'"], [15,
"'0'"], [16, "'1'"], [17, "'2'"], [18, "'3'"], [19, "'4'"]]
```

In [166]:
```
from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained("Qwen/Qwen2.5-72B-Instruc
t")
text = "What is natural language processing? 🤗"
encoded = tokenizer(text, return_tensors="pt")
print("Input IDs:", encoded["input_ids"])
# Decode including special tokens
decoded = tokenizer.decode(encoded["input_ids"][0])
print("Decoded text:", decoded)
```

```
Input IDs: tensor([[ 3838,   374,  5810,  4128,  8692,    30, 1116
2,    97,   245]])
Decoded text: What is natural language processing? 🤗
```

In [167]:
```
from transformers import AutoTokenizer
# Download vocabulary from huggingface.co and cache.
tokenizer = AutoTokenizer.from_pretrained("google-bert/bert-base-un
cased")
# Download vocabulary from huggingface.co (user-uploaded) and cach
e.
tokenizer = AutoTokenizer.from_pretrained("dbmdz/bert-base-german-c
ased")
# If vocabulary files are in a directory (e.g. tokenizer was saved
using *save_pretrained('./test/saved_model/')*)
# tokenizer = AutoTokenizer.from_pretrained("./test/bert_saved_mode
l/")
# Download vocabulary from huggingface.co and define model-specific
arguments
tokenizer = AutoTokenizer.from_pretrained("FacebookAI/roberta-bas
e", add_prefix_space=True)
```

5. Minimum Edit Distance

5.1 Algorithm analysis

Assessing the similarity between two strings is essential in many NLP tasks. For spell correction, one must choose the most similar word to a misspelled input from a set of candidates. For example, if `graffe"` is the input, among the options `graf` , `graft` , `grail` , and `giraffe` , the task is to select the word closest to the input based on a certain criterion. Similarly, comparing a translated sentence to a reference translation helps evaluate the performance of translation models in machine translation. This process may involve calculating the minimum edit distance to determine the similarity of two sentences. This note gives a dynamic programming algorithm for MED.

Definition (Minimum Edit Distance, MED).
Given two strings $X = [x_1, x_2, \ldots, x_n]$ and $Y = [y_1, y_2, \ldots, y_m]$, the minimum edit distance between X and Y is the minimum number of edit operations required to convert X into Y.

Allowed edit operations:

1. **Insertion:** add a character to X.
2. **Deletion:** remove a character from X.
3. **Substitution:** replace a character in X with another character.

These operations are applied sequentially from left to right to obtain Y.

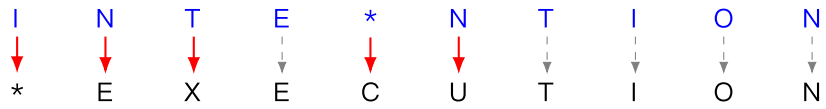
Example. Suppose $X = \texttt{INTENTION}$ and $Y = \texttt{EXECUTION}$. One possible process of transforming $X \to Y$ is:

Step-by-step edits (one possible alignment):

$$
\begin{aligned}
&1.\ \texttt{INTENTION} \xrightarrow{\text{Del}(\texttt{I})} \texttt{NTENTION} \\
&2.\ \texttt{NTENTION} \xrightarrow{\text{Sub}(\texttt{N}\to\texttt{E})} \texttt{ETENTION} \\
&3.\ \texttt{ETENTION} \xrightarrow{\text{Sub}(\texttt{T}\to\texttt{X})} \texttt{EXENTION} \\
&4.\ \texttt{EXENTION} \xrightarrow{\text{Ins}(\texttt{C})} \texttt{EXECNTION} \\
&5.\ \texttt{EXECNTION} \xrightarrow{\text{Sub}(\texttt{N}\to\texttt{U})} \texttt{EXECUTION} = Y
\end{aligned}
$$

We assume that **Ins** and **Del** each count as **1** operation, while **Sub** counts as **2** operations. Consequently, the total number of edit operations above is $1 + 2 + 2 + 1 + 2 = 8$.

Let $O_k = [o_1, o_2, \ldots, o_k]$ denote the sequence of operations transforming X into Y, where each $o_i \in \{\text{Ins}, \text{Del}, \text{Sub}\}$.
We call O_k an alignment of X into Y (see the figure below).



- **Dynamic Programming for MED**

A naive approach would examine all possible edit sequences O_k and choose the one with the minimal cost. However, this method has **exponential** time complexity. Instead, we break the problem into smaller subproblems and exploit **optimal substructure**.

Problem setting. For $i = \{0, 1, \ldots, n\}$ and $j = \{0, 1, \ldots, m\}$, let

- $X_i = [x_1, x_2, \ldots, x_i]$ be the prefix of X of length i.
- $Y_j = [y_1, y_2, \ldots, y_j]$ be the prefix of Y of length j.

By default, $X_0 = \emptyset$ and $Y_0 = \emptyset$. Define $D[i, j]$ as the **minimum edit distance** from $X_i \to Y_j$. Clearly, $D[n, m]$ is the MED of $X \to Y$. There are two base cases:

- $D[i, 0] = i$ (delete all i characters from X_i).
- $D[0, j] = j$ (insert all j characters into $\emptyset$).

In the rest, we assume $i, j \geq 1$.

Optimal substructure. Let $O_k = [o_1, o_2, \ldots, o_k]$ be a minimum-cost sequence of operations that transforms $X_i \to Y_j$. Then

$$X_i = X_i^0 \xrightarrow{o_1} X_i^1 \xrightarrow{o_2} X_i^2 \to \cdots \xrightarrow{o_{k-1}} X_i^{k-1} \xrightarrow{o_k} X_i^k = Y_j.$$

If we remove the last operation o_k, the remaining sequence $O_{k-1} = [o_1, o_2, \ldots, o_{k-1}]$ must be an optimal process for $X_i \to X_i^{k-1}$. Otherwise, suppose there exists another sequence O'_{k-1} that transforms $X_i \to X_i^{k-1}$ with strictly smaller cost: $\text{cost}(O'_{k-1}) < \text{cost}(O_{k-1})$. Then combining O'_{k-1} with the last operation o_k would yield a sequence for $X_i \to Y_j$ with total cost strictly smaller than $\text{cost}(O_k)$—a contradiction. This is the **optimal substructure** property.

Because operations proceed from the left of X_i to the right, there are only **three** possibilities for the last operation o_k:

- **Case 1 (Deletion).** Delete x_i from X_i.

$$D[i, j] = D[i-1, j] + \text{Del}(x_i).$$

- **Case 2 (Insertion).** Insert y_j into Y_{j-1}.

$$D[i, j] = D[i, j-1] + \text{Ins}(y_j).$$

- **Case 3 (Substitution).** Substitute x_i by y_j.

$$D[i, j] = D[i-1, j-1] + \text{Sub}(x_i, y_j).$$

Therefore, MED for $X_i \to Y_j$ can be computed from the three subproblems $D[i-1, j]$, $D[i, j-1]$, and $D[i-1, j-1]$ by taking the minimum:

$$D[i, j] = \min \begin{cases} D[i-1, j] + \text{Del}(x_i), \\ D[i, j-1] + \text{Ins}(y_j), \\ D[i-1, j-1] + \text{Sub}(x_i, y_j). \end{cases}$$

This recurrence is the core of **dynamic programming** for MED. In the example above, we can compute the DP matrix $\mathbf{D}$ using this recurrence, as shown in the following tables. The above formula is called dynamic procedure or dynamic programming. Given the above-mentioned example, we can calculate matrix $\mathbf{D}$ using this dynamic procedure as shown in the following tables.

- **DP Table: Initialization vs. Final Result**

Initial stage: $D[\cdot,\cdot]$

N	9									
O	8									
I	7									
T	6									
N	5									
E	4									
T	3									
N	2									
I	1									
$\#$	0	1	2	3	4	5	6	7	8	9
	$\#$	E	X	E	C	U	T	I	O	N

Final stage: $D[n,m] = 8$

N	9	8	9	10	11	12	11	10	9
O	8	7	8	9	10	11	10	9	8
I	7	6	7	8	9	10	9	8	9
T	6	5	6	7	8	9	8	9	10
N	5	4	5	6	7	8	9	10	11
E	4	3	4	5	6	7	8	9	10
T	3	4	5	6	7	8	7	8	9
N	2	3	4	5	6	7	8	7	8
I	1	2	3	4	5	6	7	6	7
$\#$	0	1	2	3	4	5	6	7	8
	$\#$	E	X	E	C	U	T	I	O

Remark. We call the above method a **dynamic programming** method.

1. Try to prove or disprove the **uniqueness** of the optimal sequence of operations.
2. Can you find approximate solutions if n and m are large? (Hint: search for approximate string matching (https://en.wikipedia.org/wiki/Approximate_string_matching).)

5.2 Algorithm implementation

In [91]:
```
# define minimum edit distance algorithm via dynamic programming
def minimum_edit_distance(source, target):
    n = len(source)
    m = len(target)
    d_mat = np.zeros((n + 1, m + 1))
    for i in range(1, n + 1):
        d_mat[i, 0] = i
    for j in range(1, m + 1):
        d_mat[0, j] = j
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            sub = 0 if source[i - 1] == target[j - 1] else 2
            del_ = d_mat[i - 1][j] + 1
            ins_ = d_mat[i][j - 1] + 1
            d_mat[i][j] = min(del_, ins_, d_mat[i - 1][j - 1] + su
b)
    trace, align_source, align_target = backtrack_alignment(source,
target, d_mat)
    return d_mat[n, m], trace, align_source, align_target

def backtrack_alignment(source, target, d_mat):
    align_source, align_target = [], []
    i, j = len(source), len(target)
    back_trace = [[i, j]]

    while (i, j) != (0, 0):
        # boundary cases first (avoid negative indexing)
        if i == 0:
            back_trace.append([i, j - 1])
            align_source = ["*"] + align_source
            align_target = [target[j - 1]] + align_target
            j -= 1
            continue
        if j == 0:
            back_trace.append([i - 1, j])
            align_source = [source[i - 1]] + align_source
            align_target = ["*"] + align_target
            i -= 1
            continue

        sub_cost = 0 if source[i - 1] == target[j - 1] else 2

        # prefer substitution/match when optimal (your tie-break ru
le)
        if d_mat[i, j] == d_mat[i - 1, j - 1] + sub_cost:
            back_trace.append([i - 1, j - 1])
            align_source = [source[i - 1]] + align_source
            align_target = [target[j - 1]] + align_target
            i, j = i - 1, j - 1

        # deletion
        elif d_mat[i, j] == d_mat[i - 1, j] + 1:
            back_trace.append([i - 1, j])
            align_source = [source[i - 1]] + align_source
            align_target = ["*"] + align_target
            i -= 1
```

```
        # insertion
        else:
            back_trace.append([i, j - 1])
            align_source = ["*"] + align_source
            align_target = [target[j - 1]] + align_target
            j -= 1

    return back_trace, align_source, align_target

# test the minimum edit distance
def test_med(source, target):
    med, trace, align_source, align_target = minimum_edit_distance
(source, target)
    print(f"input source: {source} and target: {target}")
    print(f"med: {med}")
    print(f"trace: {trace}")
    print(f"aligned source: {align_source}")
    print(f"aligned target: {align_target}")
```

In [92]:
```
test_med(source="INTENTION", target="EXECUTION")
```

```
input source: INTENTION and target: EXECUTION
med: 8.0
trace: [[9, 9], [8, 8], [7, 7], [6, 6], [5, 5], [4, 4], [4, 3], [3,
2], [2, 1], [1, 0], [0, 0]]
aligned source: ['I', 'N', 'T', 'E', '*', 'N', 'T', 'I', 'O', 'N']
aligned target: ['*', 'E', 'X', 'E', 'C', 'U', 'T', 'I', 'O', 'N']
```

In [93]:
```
test_med(source="AGGCTATCACCTGACCTCCAGGCCGATGCCC", target="TAGCTATC
ACGACCGCGGTCGATTTGCCCGAC")
```

```
input source: AGGCTATCACCTGACCTCCAGGCCGATGCCC and target: TAGCTATCA
CGACCGCGGTCGATTTGCCCGAC
med: 15.0
trace: [[31, 32], [30, 31], [30, 30], [30, 29], [29, 28], [28, 27],
[28, 26], [27, 25], [26, 24], [26, 23], [26, 22], [25, 21], [24, 2
0], [23, 19], [22, 18], [21, 17], [20, 16], [19, 16], [18, 15], [1
7, 14], [16, 14], [15, 13], [14, 12], [13, 11], [12, 10], [11, 10],
[10, 9], [9, 9], [8, 8], [7, 7], [6, 6], [5, 5], [4, 4], [3, 3],
[2, 2], [1, 2], [0, 1], [0, 0]]
aligned source: ['*', 'A', 'G', 'G', 'C', 'T', 'A', 'T', 'C', 'A',
'C', 'C', 'T', 'G', 'A', 'C', 'C', 'T', 'C', 'C', 'A', 'G', 'G',
'C', 'C', 'G', 'A', '*', '*', 'T', 'G', '*', 'C', 'C', '*', '*',
'C']
aligned target: ['T', 'A', '*', 'G', 'C', 'T', 'A', 'T', 'C', 'A',
'*', 'C', '*', 'G', 'A', 'C', 'C', '*', 'G', 'C', '*', 'G', 'G',
'T', 'C', 'G', 'A', 'T', 'T', 'T', 'G', 'C', 'C', 'C', 'G', 'A',
'C']
```

6. Submit your work

In [171]:
```
# Finally, you all done! Please submit your jupyter notebook via el
earning!

your_chinese_name = "XYZ" # put your chinese name here
your_student_id = "" # put your student id here

jnk_end_time = time.time()
total = jnk_end_time - jnk_start_time
h, rem = divmod(int(total), 3600)
m, s = divmod(rem, 60)

print(f"Total runtime: {h} h {m} min {s} s")
print(f"I am {your_chinese_name}:{your_student_id}, I am doing a gr
eat job!")
```

```
Total runtime: 2 h 38 min 23 s
I am XYZ:, I am doing a great job!
```

- **Run the following command and submit the pdf version accordingly**

In [170]:
```
!jupyter nbconvert lecture-01-exercise-tokenization.ipynb --to html
--template classic --embed-images
```

```
[NbConvertApp] Converting notebook lecture-01-exercise-tokenizatio
n.ipynb to html
[NbConvertApp] WARNING | Alternative text is missing on 5 image(s).
[NbConvertApp] Writing 13707308 bytes to lecture-01-exercise-tokeni
zation.html
```

7. Useful commands

- **1. Save your jupyter notebook as a PDF file**

To save your notebook as pdf you can use the following command

```
jupyter nbconvert lecture-01-exercise-tokenization.ipynb --to html --template
classic --embed-images
```

- **2. Save your env from installed history**

```
conda env export --from-history > your-environment.yml
```

If you do not want to have prefix in the end of yml file, you can try to remove the prefix line by using:

```
conda env export --from-history > your-environment.ymlsed -i '' '/^prefix: /
d' environment.yml   # macOS sed
```

- **3. Start uvicorn server**

```
uvicorn test_sd_server:app --host 127.0.0.1 --port 5055
```